Vrije Universiteit Amsterdam

Universiteit van Amsterdam

Master Thesis

Formal Verification of Fault-Tolerant Safra's Algorithm

Author: Andy S. Tatman (2818888)

1st supervisor: Prof. Dr. W.J. Fokkink
2nd reader: Dr. A. Mehrabi

A thesis submitted in fulfilment of the requirements for the joint UvA-VU Master of Science degree in Computer Science

January 26, 2026

Abstract

In this work, we formally verify the correctness of a fault-sensitive and a fault-tolerant version of Safra's algorithm for termination detection. We do this by providing a paper proof of the Liveness property for the fault-sensitive algorithm, as well as a proof sketch for the same property in the fault-tolerant algorithm. Next, we identify a bug in the fault-tolerant algorithm, and propose a solution.

From the fault-sensitive algorithm and the adjusted fault-tolerant algorithm, we create formal specifications, and use these to generate a variety of models. We separately specify a number of properties that should hold for the two algorithms, and use model checking to verify that these properties hold for our various models.

Contents

1

Introduction

Writing a programme that runs across multiple computers is a famously difficult task, as this throws up problems that do not occur when running on a single device. As such, we wish to ensure that, before we start writing code, the underlying algorithm we use works as intended.

In (37), a new version of Safra's algorithm was proposed, where we can detect termination in a distributed setting even if one or more nodes in the system crash.

In this work, we set out to perform formal verification, through model checking, of both (37) and (12), a version of Safra's algorithm that does *not* support crashes and from which (37) was created.

Specifically, we set out the following goals:

1. To create formal specifications of the fault-sensitive (12) and fault-tolerant (37) algorithms, using the mCRL2 language (25).

2. To specify properties that should hold for the underlying algorithms (in words), and then specify these properties for our mCRL2 models. Specifically, we intend to verify the two properties explicitly mentioned in (37), as well as any other relevant properties we encounter.

3. To use the mCRL2 toolset (4) to create specific model instances, based on our specifications, and to check our properties on these models.

4. Finally, if properties do *not* hold, we set out to identify and correct any bugs in the underlying algorithms.

In this work, we identify one bug in the fault-tolerant algorithm, both through an example execution and through our formal model, and we set out a simple change that addresses the bug.

Working with this corrected algorithm, we provide a paper proof of the Liveness property for arbitrary instances, and we create a formal specification and series of models of the two algorithms for a variety of instances. With these models, we then show that the various properties that we set out hold in our models.

In Chapter 2, we provide some background on both distributed algorithms, specifically also detailing our two algorithms, and on formal verification, specifically discussing our chosen tools, with Chapter 3 discussing some related work in this field.

Chapter 4 then sets the approach we take, details our paper proofs justifying part of this approach, and we provide our list of properties (in words) which we set out to check. In Chapter 5, we then detail our formal specifications for the two algorithms, and provide formulas for each of our properties, which we can use to check our models.

In Chapter 6 we detail how we generate and check our models with the mCRL2 toolset (4), as well as list the various the size of our models and some times for checking these properties. We discuss our results in Chapter 7, and here we also discuss the bug in the fault-tolerant algorithm along with our proposed solution.

All of our specifications, formulas and generated files are available on Zenodo (58). Alternatively, we provide just the specifications, as well as the pythons scripts used to generate many of the files, in our GitHub repository, also linked in (58).

2

Background

2.1 Distributed algorithms

Classically, solutions to problems in computer science have been presented with a single execution thread. However, in order to work on a larger-scale, we can make use of distributed computing: instead of a single machine working on the problem, we can have a series of nodes connected in a network all working on the same problem. As an example, the Travelling Salesperson problem rapidly grows beyond what a single computer is capable of handling efficiently (35).

These distributed systems may not always be able to directly share memory. Where different parts of the system are far apart, e.g. (5), this is infeasible. Instead, we work in the message-passing paradigm, where nodes are connected by channels through which they can send each other messages.

Running a programme in a distributed setting produces unique challenges compared to a uniprocessor setting. As an example: it is easy to determine the state of the programme when working with a single execution thread. However, when working with multiple nodes and in the absence of a global clock, it takes considerably more effort just to get a consistent view of the current system (7).

We will focus on one such problem here: Termination Detection, which is addressed by the algorithms we analyse in this work.

2.1.1 Setting

First, we set out some definitions for working in a distributed setting.

We work with a number of distinct **nodes**. Each node may be an individual computer, or part of a larger server. Individual nodes may be connected to each other by a network,

through which they can send each other **messages**, such as for exchanging data.

The **basic algorithm** is the main computation that the system is actively working on, such as an algorithm trying to solve the travelling salesperson problem. A node is either **active**, so currently working on the basic algorithm, or **passive**, meaning that it is currently finished with its work and might be waiting on some incoming message before it can continue. We say that the basic algorithm has **terminated** when 1) all nodes are passive and when 2) there are no basic messages travelling between nodes[1]. In this thesis, we assume that once the basic algorithm has terminated across the entire network, the basic algorithm will remain terminated [2].

We then have one or more **control algorithms** which work on addressing underlying problems, such as detecting when the basic algorithm has terminated. As with the algorithms, we distinguish between **basic** and **control messages**.

A node may **crash**, meaning that it immediately stops any computation it may be working on (basic or control), and no longer can send or receive messages. Under our assumptions (see Section 2.1.3.1), we assume that such crashes are permanent.

2.1.2 Termination detection

Given some basic algorithm, which is performing a computation on a set of nodes, the goal of a termination detection (control) algorithm is to determine when this basic algorithm has terminated. When some node detects that the network has terminated, we say that the node **Announces** this.[3]

The termination detection problem was originally introduced in (16, 20). Complications arise from the fact that, even if all nodes are passive at some point in time, there still be some basic message travelling between nodes, which will thus wake up the receiving node.

A number of different algorithms have been presented which tackle this problem, which we will discuss in Chapter 3. Below, we discuss Safra's algorithm, which is the primary focus of this thesis' efforts.

For any termination detection algorithm, we have 2 properties which we seek to ensure (41) (using the naming conventions from (37)):

[1] This definition becomes more nuanced when we allow nodes to crash, see Section 2.1.3.3.

[2] This is specified as a separate property *Quiescence* in (39).

[3] We abstract away from any specific implementation, and so we leave open the question of what it means when a node Announces. To provide an example: in the experimental implementation provided in (37), Announce causes a message to go around the ring, informing nodes of the termination being detected.

- **Liveness**: If the basic algorithm has terminated, this termination is eventually detected by a (non-crashed) node.
- **Safety**: When termination is detected, the basic algorithm has indeed terminated at some point in the past.

2.1.3 Safra's algorithm

Safra's algorithm is a termination detection algorithm for nodes *arranged in a ring*, and was originally presented in (15, 18), with further optimisations presented in (12).

Both the original and optimised versions of Safra's algorithm do not allow for crashes (see Section 2.1.3.3). (37) sets out an adjusted version, based on (12), that *does* support crashing nodes. We refer to the version that does *not* allow for crashes as being **fault-sensitive**, while we refer to the version that does support crashing nodes as **fault-tolerant**.

2.1.3.1 Assumptions

As mentioned above, we work in the message passing paradigm. Specifically, we assume that there is no shared memory between nodes, and no global clock (7). A message that is sent will always eventually arrive at the destination node (assuming the destination node does not crash prior to arrival), meaning that messages are never lost and have "unbounded but finite delays" (37) before arrival. Messages may arrive out of order.

Safra's algorithm assumes that we have a ring of $N \geq 2$ nodes. While we could have an *actual* ring, as long as each node has a unique ID, and the IDs are totally-ordered, we can work with a *logical* ring where a node i knows that $(i+1)\%N$ is next in the ring. For our purposes, we assume that we have nodes labelled with IDs 0, 1, ..., $(N-1)$.

Regarding the basic algorithm, a node that becomes/is passive can only become active when this node receives a basic message.

When a node Announces that it has detected termination, we will stop executing the procedure.[1]

We assume that, aside from crashing (see below), nodes execute the control algorithms correctly as specified, and thus do not exhibit Byzantine faults (19, 51).

Below, we will set out a series of code blocks that we execute to run Safra's algorithm (either version). With the exception of the $wait(passive_i)$ calls, we execute these procedures in their entirety without interruption, meaning for example that we cannot receive

[1] This approach is also taken in (37): "If no termination is detected, i" continues running the ReceiveToken$_i$ procedure.

a basic message (or we must buffer this incoming message) while working on the control algorithm.

For the fault-sensitive version of Safra's algorithm (see Section 2.1.3.2), we do not allow nodes to crash, and the only requirement on the network topology is that for each node i, it must be possible to send control messages to node $(i+1)\%N$.

For the fault-tolerant version of Safra's algorithm (see Section 2.1.3.3), nodes *can* crash. Any nodes that crashes must remain crashed permanently, meaning that they cannot recover or rejoin the network, and we do not allow for nodes to join the network after the start of the execution. We also require a fully connected network (48), meaning that each node i can send messages to each other node j, where $i \neq j$. For each pair of nodes i, j, it is possible that *all other nodes crash*. For termination to be detected, we must then be able to communicate between i and j. In order for a node i to find out that a node j has crashed, we require a *perfect* failure detector (6). This means that only crashed nodes are ever flagged as having crashed. and that each non-crashed node will eventually be informed that j has crashed. Finally, for purposes of verification, we assume that always at least 1 node remains non-crashed, as termination detection is trivial when all nodes have crashed.

2.1.3.2 Fault-sensitive Safra's algorithm, with optimisations

We will discuss Safra's algorithms, where we do *not* allow nodes to crash, based on the optimisations presented in (12).

In its essence, Safra's algorithm works by having passive nodes pass a token, containing some data, the ring. Each node keeps track of 3 values for the algorithm: $count_i$, which stores the number of basic messages *sent* minus the number of basic messages the node has *received*; $black_i$, which we will discuss in more detail shortly; and seq_i, which keeps track of the number of times the iterations of the ring that this node has carried out. In Algorithm 1, we initialise these values.

Algorithm 1 Initialisation$_i$

1: $count_i \leftarrow 0$; $black_i \leftarrow i$; $seq_i \leftarrow 0$;
2: **if** $i = 0$ **then**
3: $wait(passive_0)$
4: $count_t \leftarrow count_0$; $black_t \leftarrow N-1$; $send(t, 1)$; $count_0 \leftarrow 0$; $seq_0 \leftarrow 1$;

When a node sends a basic message, we attach to this message the sequence counter seq_i. We also increment the local counter $count_i$. (See Algorithm 2.)

Algorithm 2 SendBasicMessage$_i(m, j)$

1: $seq_m \leftarrow seq_i$; $send((m, j))$; $count_i \leftarrow count_i + 1$;

We eventually want to call termination when the token's counter $count_t = 0$. However, take the example where node 0 sends one message and receives one message. Here, $count_0 = 0$, but clearly we cannot yet determine whether termination has occurred in the basic algorithm, as there is at least one other node that might be active (namely, the one that sent a basic message to 0). To solve this issue, Safra's algorithm uses the $black_i$ variable. Originally, only node 0 could determine termination, and a node i receiving any message would colour the node black. When the token arrives at a black node, the token in turn is coloured black, meaning that the token would need to visit node 0 at least 2 more times before we could determine termination.

In (12)'s version, we lift both of these restrictions: *any* node can now detect termination. Using $black_i$, we indicate to the token how far in the ring it must travel before it is safe to detect termination. In Initialisation$_i$ 1, we set $black_t$ to $N - 1$, meaning that we guarantee that the token visits all nodes in the ring at least once before we can detect termination. When i receives a basic message m from j, we examine the sequence counter seq_m attached to the message. If the message was sent after the token passed j and before i has handled the token ($seq_m = seq_i + 1$), or if j is further in the ring than i and the token is not already on its way to j from i ($j > i \wedge seq_m = seq_i$), then we update $black_i$.[1] See Algorithm 3.

Algorithm 3 ReceiveBasicMessage$_i(m, j)$

1: **if** $seq_m = seq_i + 1 \vee (j > i \wedge seq_m = seq_i)$ **then**
2: $\quad black_i \leftarrow furthest_i(black_i, j)$
3: $count_i \leftarrow count_i - 1$;
4: ▷ If node i was passive previously, it now becomes *active*.

Definition 2.1.1 ($furthest_i$)**.**

$$furthest_i = \begin{cases} k & \text{if } i \leq j \leq k \\ k & \text{if } k < i \leq j \\ k & \text{if } j \leq k < i \\ j & \text{otherwise} \end{cases}$$

[1] We assume (and will verify) that $seq_m \leq seq_i + 1$. See Section 4.3.

As the name implies, $furthest_i(j,k)$ returns whichever value is further around the ring, relative to node i's position.

This now brings us to the final part of Safra's algorithm, where we handle the actual token. We start executing Algorithm 4. Once the node is passive, we add $count_i$ to the token's counter $count_t$, and (potentially) update $black_i$. If $count_t = 0$ and $black_i = i$, then we have found that the basic algorithm has terminated, and we can Announce. Else, we update $black_t$, as the token must now at least go to the next node $(i+1)\%N$, and then we send the token on. After passing the token on, we reset the local $count_i$ and $black_i$ values, and increment the sequence counter.

Algorithm 4 ReceiveToken$_i$

1: $wait(passive_i)$
2: $count_t \leftarrow count_t + count_i$; $\quad black_i \leftarrow furthest_i(black_i, black_t)$;
3: **if** $count_t = 0 \wedge black_i = i$ **then**
4: Announce;
5: $black_t \leftarrow furthest_i(black_i, (i+1)\%N)$;
6: $send(t, (i+1)\%N)$;
7: $count_i \leftarrow 0$; $\quad black_i \leftarrow i$; $\quad seq_i \leftarrow seq_i + 1$;

Example execution

We provide an example execution of this algorithm, where we have three nodes 0, 1 and 2:

- Initially, $seq_i = 0$, $count_i = 0$, and $black_i = i$ for all three nodes i.
- 0 sends a basic message m_1, tagged with $seq_{m_1} = 0$, to node 1. $count_0 = 1$.
- 0 becomes passive. Following Initialisation$_0$ 1, we set $count_t = count_0 = 1$ and $black_t = 2$. As such, we send $t = \langle count_t, black_t \rangle = \langle 1, 2 \rangle$ to node 1.
- 1 and 2 also become passive. They do not (yet) hold the token, thus the nodes take no action (yet).
- Node 1 receives the token. $count_t$ is unchanged, while $black_1 = furthest_1(1,2) = 2$. $count_t \neq 0$, so we continue. $black_t = furthest_1(2,2) = 2$. Node 1 sends the token $t = \langle 1, 2 \rangle$ to node 2. $seq_1 = 1$, and $count_1 = 0$.
- Node 2 receives the token. $count_t$ and $black_2 = furthest_2(2,2) = 2$ are again unchanged. $count_t \neq 0$, so we continue. $black_t = furthest_2(2,0) = 0$. Node 1 sends the token $t = \langle 1, 0 \rangle$ to node 2. $seq_2 = 1$.

Note at this point that, as long as one or basic messages do not arrive at their node, we can simply continue sending the token around the ring. See Section 4.1.2 where we discuss this further.

- Node 1 receives m_1. $seq_m < seq_1$, so we do not update $black_1$. $count_1 = -1$, and 1 becomes active.
- Node 1 sends m_2, tagged with $seq_{m_2} = 1$, to node 1, $count_1 = 0$. Node 1 becomes passive again.
- Node 0 receives m_2. $(1 > 0) \ \wedge \ seq_{m_2} = seq_0$ is true, so we update $black_0 = furthest_0(0, 1) = 1$, and we set $count_0 = -1$. Node 0 becomes active upon receiving the basic message, and we immediately let it become passive again.
- At this point, the basic algorithm has terminated, as all basic messages have been received and all nodes are passive.
- Node 0 receives the token. $count_t = count_t + count_0 = 1 + (-1) = 0$. $black_0 = furthest_0(1, 0) = 1$. As $black_0 \neq 0$, we continue. $black_t = furthest_0(1, 1) = 1$, so 0 sends the token $\langle 0, 1 \rangle$ to node 1. $seq_0 = 2$, and we reset $black_0 = 0$.
- Node 1 receives the token. $count_t$ and $black_1 = furthest_1(1, 1) = 1$ are unchanged. $count_t = 0 \ \wedge \ black_1 = 1$, so node 1 Announces that it has detected termination.

2.1.3.3 Fault-tolerant Safra's algorithm

Here, we will discuss the altered version of Safra's algorithm discussed in (37), in a setting where any arbitrary amount of nodes $< N$ in the network may to crash.

Altered definition of termination

First, we need to adjust our definition of when the basic algorithm has terminated. Following the definition used in (37), the basic algorithm has terminated when 1) all nodes are passive *or crashed* and when 2) for any basic messages travelling from a node i to a node j, either j has crashed or j knows that i has crashed. This definition implies the original condition, as there can be no basic messages travelling between pairs of *non*-crashed nodes. For any fault-tolerant algorithm, if j knows that i has crashed, j should *not* become active on receiving a basic message from i if there is a chance that we detect termination before j becomes passive again.[1]

[1] In Section 7.1, we discuss this expectation being violated.

Why are adjustments necessary?

Next, we set the scene for why we need to adjust the algorithm previously discussed. We require a perfect failure detector in order to carry out termination detection when nodes can crash (29, 48), so let us assume that we have one.

Say we have a ring of $N = 3$ nodes, and node 0 sends a basic message to node 1, and let node 1 crash before this basic message arrives. Now let nodes 0 and 2 become passive. At this point, the basic algorithm has terminated: there are no active nodes, and there are no messages travelling to any live nodes.

The first issue that would arise from the previous algorithm, is that 0 will try and send the token to 1. Given that 1 has crashed, 1 will never forward the token on to 2. As such, one adjustment we require is some method of re-routing (or re-sending) the token when we detect a node has crashed.

Now, let us assume we have such a re-routing mechanism. 0 sent a basic message, meaning that we will set $count_t$ to 1 as $count_0 = 1$. However, as 1 never received either the token or the basic message before crashing, 1 will never apply the matching decrement to $count_t$. As such, we have two nodes who are both passive and both have $black_i = i$, but $count_t$ will never become 0, so termination will never be found. 0 simply remembering that it sent 1 one message, and removing the +1 from the token is also not necessarily a solution [1]. Instead, the solution presented in (37) is to keep track of a list of counters for each node, both in the token and at each node.

The fault-tolerant algorithm

We now detail the algorithm as laid out in (37). Each node now keeps track of a variable $next_i$ which indicates which node is next in the ring, passing over any crashed nodes. Each node also keeps track of a list of counters $count_i$, where $count_i^j$ stores the difference between messages i sent to j and those sent from j to i.

We keep track of the nodes that have crashed in the (local) sets $Crashed_i$ and $Report_i$, and in the token in $Crashed_t$. We will discuss these in more detail further on.

[1] Take another scenario, where 1 *did* receive the basic message & token, and passed the token on to 2 before crashing, with for example 2 also sending some basic message. 0 decrementing $count_t$ after node 1's crash could similarly here result in termination never being found.

Algorithm 5 Initialisation$_i$

```
1: for j = 0 to N − 1 do
2:     count_i^j ← 0;   count_j^t ← 0;
3: black_i ← i;   seq_i ← 0;   next_i ← (i + 1)%N;
4: Crashed_i ← ∅;   Crashed_t ← ∅;   Report_i ← ∅;
5: if i = 0 then
6:     black_t ← N − 1;   seq_t ← 1;   ReceiveToken_0;
7: else
8:     black_t ← i;
```

The SendBasicMessage$_i(m, j)$ procedure in Algorithm 6 is very similar to that in fault-sensitive version in Algorithm 2. We only allow i to send basic messages to nodes j if, to the i's current knowledge, j is not crashed. The set $Crashed_t$ refers to the set attached to a token that has arrived and is currently being buffered (as i is active), and thus has not been dismissed.

Algorithm 6 SendBasicMessage$_i(m, j)$

```
1: if j ∉ Crashed_i ∪ Report_i ∪ Crashed_t then
2:     seq_m ← seq_i;   send(m, j);   count_i^j ← count_i^j + 1;
```

Similarly, ReceiveBasicMessage$_i(m, j)$ in Algorithm 7 is very close to its fault-sensitive counterpart in Algorithm 3. Notably, (37) specifically emphasises that a node i *can* receive messages from nodes $\in Report_i$. Similarly, unlike Algorithm 6, we *can* receive messages from nodes in $Crashed_t$.

Algorithm 7 ReceiveBasicMessage$_i(m, j)$

```
1: if j ∉ Crashed_i then
2:     if seq_m = seq_i + 1 ∨ (j > i ∧ seq_m = seq_i) then
3:         black_i ← furthest_i(black_i, j)
4:     count_i^j ← count_i^k j − 1;
5:     ▷ If node i was passive previously, it now becomes active.
```

Next, we turn to the procedure that we run when the failure detector finds that a node j has crashed. If i was not already aware of j having crashed, we add j to $Report_i$. If j was the next alive node in the ring, then i needs to update $next_i$ to find the new node to which we send the token, thereby effectively removing crashed node(s) from the ring. If

we have already sent one or more tokens on ($seq_i > 0$), then we resend the previous token to the new $next_i$, as the token might have been lost when node j crashed. Alternatively, if $seq_i = 0$ but $next_i < i$ is true, then the node that should have started the first iteration of the algorithm may have crashed before sending out the first token. [1] In this case, we also send out an extra token, to ensure that the first iteration is indeed started. If we start a new iteration of the ring $next_i < i$, then we increment the token's sequence counter.

Algorithm 8 FailureDetector$_i$

```
1:  crashed(j);
2:  if j ∉ Crashed_i ∪ Report_i then
3:      Report_i ← Report_i ∪ {j};
4:      if j = next_i then
5:          NewSuccessor_i;
6:          if seq_i > 0 ∨ next_i < i then
7:              Crashed_t ← Crashed_t ∪ Report_i;
8:              black_t ← i;
9:              if next_i < i then
10:                 seq_t ← seq_i + 1;
11:             send(t, next_i);
```

Now, we define NewSuccessor$_i$ in Algorithm 9. When this procedure is called, we know that $next_i$ is a crashed node. Before we can thus continue, we need to find a new target to which we can send the token. This function also includes a break case: if $next_i = i$, then node i is the only node in the network that has not yet crashed. In that case, termination detection is trivial: once this one node is passive, we know that we can Announce. [2] If we are *not* in this break case, then we update $black_i$ to ensure that at a minimum, the token will go to $next_i$ before we can check for termination.

[1] This happens when node 0 crashes before calling ReceiveToken$_0$ in Initialisation$_0$ 5, or before sending out the first token.

[2] Note that if $next_i = i$, that means that we have passed every other value in the ring, and found it in $Crashed_i \cup Report_i$. As such, $Crashed_i \cup Report_i$ should equal $\{1, .., N\} \setminus \{i\}$.

Algorithm 9 NewSuccessor$_i$

Require: $\exists j \in [0..N-1].j \notin \mathit{Crashed}_i \cup \mathit{Report}_i$

1: $\mathit{next}_i \leftarrow (\mathit{next}_i + 1)\%N$; ▷ Initially, $\mathit{next}_i \in \mathit{Crashed}_i \cup \mathit{Report}_i$
2: **while** $\mathit{next}_i \in \mathit{Crashed}_i \cup \mathit{Report}_i$ **do**
3: $\quad \mathit{next}_i \leftarrow (\mathit{next}_i + 1)\%N$;
4: **if** $\mathit{next}_i = i$ **then**
5: $\quad \mathit{wait}(\mathit{passive}_i)$;
6: $\quad$ Announce;
7: **if** $\mathit{black}_i \neq i$ **then**
8: $\quad \mathit{black}_i \leftarrow \mathit{furthest}_i(\mathit{black}_i, \mathit{next}_i)$;

Finally, we come to ReceiveToken$_i$, which is considerably more involved than the version in Algorithm 4. We show this procedure in Algorithm 10. First of all, we check seq_t. If seq_t does *not* equal $\mathit{seq}_i + 1$, then the token is not the token for the following iteration of the control algorithm, and we dismiss the token. This should only occur when we receive an old token, for example if a duplicate token is sent after some node has crashed. (See Section 4.3.)

If we do decide to handle this token, then we buffer the token until the node becomes passive. We then start updating our sets: we remove any nodes from $\mathit{Crashed}_t$ that are already in $\mathit{Crashed}_i$, as node i will have already passed these nodes on in a previous iteration. Any remaining nodes in $\mathit{Crashed}_t$ are then added to $\mathit{Crashed}_i$, and we remove any such nodes from Report_i. Skipping ahead to lines 21-22, if we do *not* Announce, then we will add all nodes currently in Report_i to the token, so that the rest of the network can be informed that these nodes have crashed, after which we add these nodes to $\mathit{Crashed}_i$ and reset Report_i.

As before, we can only detect termination at this node if $\mathit{black}_i = i$. Hence why, in line 10, we only check sum_i if $\mathit{black}_i = i$. If $\mathit{Report}_i \neq \emptyset$, we set black_t to i, meaning that the token should be *guaranteed* to return to this node [1]. The algorithm only updates count_t^i when there is a chance that we will find termination within the next iteration (without returning to node i), namely when the condition on line 6 holds.

We now *only* use the values in count_i and count_t of nodes $j \notin \mathit{Crashed}_i$. Thereby, we solve the issue we mentioned above: we can specifically exclude counter values for crashed nodes, and thus the resulting sum_i *only* includes nodes that (to i's current knowledge)

[1] With some slight exceptions, see Section 4.3.

are not crashed. Given that i will eventually be informed about *all* crashed nodes by the *perfect* failure detector, eventually i will only look at the counters from non-crashed nodes.

If we do not Announce, then we need to send on the token. If i is informed via the token that $next_i$ has crashed, then we call NewSuccesor$_i$ to update $next_i$ or apply the break case.

Similar to NewSuccesor$_i$, if the token starts a new iteration of the ring $next_i < i$, then we increment the token's sequence counter.

As in the fault-sensitive version of ReceiveToken$_i$ 4, if we do *not* have any specific target to send the token to, then we simply set $black_t$ to $next_i$, to indicate that it at least needs to reach $next_i$.

In the last line, we increment seq_i. Any duplicate tokens that arrive with the same seq_t as the token we have just processed, will thus be dismissed as duplicates.

Algorithm 10 ReceiveToken$_i$

```
1: if seq_t = seq_i + 1 then
2:     wait(passive_i);   black_i ← furthest_i(black_i, black_t);
3:     Crashed_t ← Crashed_t \ Crashed_i;
4:     Crashed_i ← Crashed_i ∪ Crashed_t;
5:     Report_i ← Report_i \ Crashed_t;
6:     if black_i = i ∨ Report_i = ∅ then
7:         count_t^i ← 0;
8:         for all j ∈ [0..N − 1] \ Crashed_i do
9:             count_t^i ← count_t^i + count_i^j;
10:    if black_i = i then
11:        sum_i ← 0;
12:        for all j ∈ [0..N − 1] \ Crashed_i do
13:            sum_i ← sum_i + count_t^j;
14:        if sum_i = 0 then
15:            Announce;
16:    if next_i ∈ Crashed_t then
17:        NewSuccessor_i;
18:    if next_i < i then
19:        seq_t ← seq_t + 1;
20:    if Report_i ≠ ∅ then
21:        Crashed_t ← Crashed_t ∪ Report_i;   black_t ← i;
22:        Crashed_i ← Crashed_i ∪ Report_i;   Report_i ← ∅;
23:    else
24:        black_t ← furthest_i(black_i, next_i);
25:    send(t, next_i);
26:    black_i ← i;   seq_i ← seq_i + 1;
27: else
28:    ▷ If seq_t ≠ seq_i + 1, then the incoming token is dismissed. It should then be the
       case that seq_t < seq_i + 1, see Section 4.3.
```

Example execution

We now provide the same example execution as we did in Section 2.1.3.2, except we now let node 1 crash just before it Announces. We leave out seq_t from our token here, as it is not used in our example.

- Initially, $seq_i = 0$, $count_i^j = 0$, and $black_i = i$ for all three nodes i and j. Furthermore,

$Crashed_i = Report_i = Crashed_t = \emptyset$, $count_j^t = 0$, and $next_i = (i+1)\%N$.

- 0 sends a basic message m_1, tagged with $seq_{m_1} = 0$, to node 1. $count_0^1 = 1$.
- 0 becomes passive. Following Initialisation$_0$ 5, we set $black_t = 2$ and $seq_t = 1$, and run ReceiveToken$_0$.
  We set $black_0 = furthest_0(0, 2) = 2$. Because $Report_0 = \emptyset$, we update $count_t$, by setting $count_t^i = 1$. $black_0 \neq 0$, so we continue by sending the token $\langle [1, 0, 0], 2\rangle$ to node 1. We reset $black_0 = 0$, and set $seq_0 = 1$.
- 1 and 2 also become passive. They do not (yet) hold the token, thus the nodes take no action (yet).
- Node 1 receives the token. $count_t$ is unchanged, while $black_1 = furthest_1(1, 2) = 2$. $black_t$ and $count_t$ are not updated. As $black_1 \neq 1$, we continue, and send the token $\langle [1, 0, 0], 2\rangle$ to node 2, after which we set $black_1 = 1$ and $seq_1 = 1$.
- Node 2 next receives the token. $count_t$ and $black_2 = 2$ are unchanged. As $sum_2 \neq 0$, we continue. $next_2 < 2$, so we set $seq_t = 2$. $black_t = furthest_2(2, 0) = 0$, so we send the token $\langle [1, 0, 0], 0\rangle$ to node 0. $black_2 = 2$, and $seq_2 = 1$.
- Node 1 receives m_1 (as $0 \notin Crashed_1$). $seq_m < seq_1$, so we do not update $black_1$. $count_1^0 = -1$, and 1 becomes active.
- Node 1 sends m_2, tagged with $seq_{m_2} = 1$, to node 1, $count_1^0 = 0$. Node 1 becomes passive again.
- Node 0 receives m_2. $(1 > 0) \wedge seq_{m_2} = seq_0$ is true, so we update $black_0 = furthest_0(0, 1) = 1$, and we set $count_0^1 = 1 - 1 = 0$. Node 0 becomes active upon receiving the basic message, and we immediately let it become passive again.
- At this point, the basic algorithm has terminated, as all basic messages have been received and all nodes are passive.
- Node 0 receives the token. $black_0 = furthest_0(1, 0) = 1$. As $Report_0 = \emptyset$, we update $count_t^0 = 0$. As $black_0 \neq 0$, we continue on. $black_t = furthest_0(1, 1) = 1$ remains unchanged, so we send the token $\langle [0, 0, 0], 1\rangle$ to node 1. $seq_0 = 2$, and $black_0$ remains $= 0$.
- In our previous example, node 1 received the token and Announced here. Instead, we will let node 1 crash.

- Node 2 detects node 1's crash. Running FailureDetector$_2$, we add j to $Report_2$. $1 \neq next_2$, so we are done.
- Similarly, node 0 detects 1's crash. Running FailureDetector$_0$, we add j to $Report_0$. As $1 = next_0$, we call NewSuccessor$_0$, and update $next_0 = 2$. Node 0 sets $black_t = 0$ and adds 1 to $Crashed_t$, and then sends out the token $\langle [0,0,0], 0 \rangle$ to 2.
- Node 2 receives this token. Node 2 sets $Crashed_2 = \{1\}$ and $Report_2 = \emptyset$. As $black_2 = furthest_2(2,0) = 0 \neq 2$, node 2 sends on the token $\langle [0,0,0], 1 \rangle$ to node 0.
- Finally, node 0 receives the token. Again, we set $Crashed_0 = \{1\}$ and $Report_0 = \emptyset$, as $1 \in Crashed_t$. Because $black_0 = 0$, we check sum_0 and we find that it equals 0. As a result, node 0 Announces that it has detected termination.

A note on $Crashed_i$ vs. $Report_i$

When a node i is informed by the failure detector that a node j has crashed (and i was not already aware of this), j is added to $Report_i$. In (37), the authors twice refer to i knowing that some other node j has crashed: "It is therefore required that passive nodes never become active by the receipt of a basic message sent by a node **they know has crashed**." and "...for all basic messages in transit, the destination node either has crashed or **knows that the sender has crashed**."

Clearly, $j \in Crashed_i$ means that i knows that j has crashed, and we treat is as such. However, when $j \in Report_i \wedge j \notin Crashed_i$,[1] the algorithm still largely treats j as though it has not crashed: i may still receive messages from j (see ReceiveBasicMessage$_i$ 7), and $count_t^j$ is not explicitly excluded when determining if $sum_i = 0$ (Line 12 of ReceiveToken$_i$ 10).

As such, we can consider nodes $j \in Report_i \wedge j \notin Crashed_t$ to be in an intermediate state: i is *aware* of j's crash, but we largely pretend that i does not yet *know* that j has crashed.[2]

[1] It appears that $Crashed_i \cap Report_i = \emptyset$ is an invariant for this algorithm, for all nodes i. We leave any verification of this invariant for future work.

[2] This has clear advantages for the underlying basic algorithm: i can still work with the last messages j sent out before crashing, as long as i has not yet moved j to $Crashed_i$.

2.2 Formal Verification

When algorithms are first put forward, authors often provide some intuition to give an indication why they believe the algorithm to be correct. The authors may instead choose to provide a more rigorous proof on paper, also known as a paper proof. Such proofs rely on the reader believing the proof, and on the author not making mistakes. Instead, we can apply more formal methods to construct our proofs, thereby increasing the reliability of both the proofs and the underlying algorithms.

2.2.1 Deductive verification

Deductive verification (27) involves specifying the algorithm in some mathematical form (the exact form depends on your chosen method & tool), and then specifying properties for this algorithm, often in the form of contracts ('Given this pre-condition, we expect this post-condition to hold after the algorithm terminates'), theorems, or invariants ('this property should always hold / should hold initially and whenever a procedure terminates'). Using a tool such as Coq (64) or Isabelle/HOL (50), the user, together with the tool, can keep applying proof steps to their proof goals until they have successfully shown that the property always holds for the algorithm.

However, while deductive verification is powerful, it requires a lot of effort to verify even a small portion of code. As an alternative, model checking is used, which is considered to be more practical than deductive verification (9, 23).

2.2.2 Model Checking

In model checking, we write a formal specification of the system or algorithm we wish to verify, often abstracting away certain details, such as how specific concepts are implemented given specific hardware.

Given such a model, users can specify properties that they want to check, often in modal logic. A model checker such as mCRL2 (4) or TLC (part of the TLA$^+$ toolset) (40) can then be used to confirm whether this property holds for this model. If the property does *not* hold, the model checker can often provide a counterexample, consisting of a trace through the model to a state or set of states where the property does not hold. The user can then use this either to debug their model or to show a flaw in the algorithm being checked.

Generally speaking, there are two types of properties to check (41): a **safety property** specifies that some unwanted behaviour never happens, while a **liveness property**

specifies that some good thing eventually always must happen. (See also the examples in 2.1.2.)

Finite model checking

In order to model check the specifications with our chosen toolset, we require *finite* models. As such, we set limits on our specifications, and thereby our models. (See Section 4.1.)

Given a state space that is either infinite or finite but too large to efficiently check explicitly, we still have a number of options in order to do *some* verification.

Firstly, we can use bounded model checking to verify inductive invariants for the system (39).

Next, we can generate a subset of the state space, and verify properties on this. The most famous technique is bitstate hashing (32): each time we generate a state, we look at the hash value. If we have multiple states with the same hash value, then we only take one of these states.

The toolset we have used, mCRL2, allows us to use a similar method, called highway search (17). The state space is generated using breadth-first-search, but we only take a limited number of states from each level. By combining this with a limit on the total number of states we generate, we can generate large state spaces and check properties on these.

With a subset of the state space, we can check *safety* properties, as we can check that some unwanted behaviour never occurs in these states. However, we cannot check *liveness* properties with a subset of states, as the 'good thing that must eventually happen' might occur in the states outside our subset.

2.2.3 mCRL2

mCRL2 is both a specification language (25) and a toolset (4), allowing users to create specifications of concurrent systems. With these specifications, mCRL2 can generate the resulting state space, and model check properties.

We will provide a relatively informal introduction to mCRL2, by discussing some small examples of both models and formulas to check properties with. For a more in-depth introduction to mCRL2, we refer to (25), while (23) provides a tutorial for model checking various simple distributed algorithms in mCRL2. Further, (24) provides a number of guidelines to avoid/limit the state space explosion problem (10) in the resulting models. For an introduction to the Parameterised Boolean Equation Systems, or PBESs, that mCRL2 creates to check the properties, we refer to Chapter 14 of (25).

2.2.3.1 Model specification

Example 1

We discuss some of mCRL2's specification language (25) using an example:

```
act
 sToChan, rToChan, cToChan: Nat;
 sFromChan, rFromChan, cFromChan: Nat;
 errorAction;

proc
 A(n:Nat) =
  sToChan(n) . A( (n+1) mod 2);

 B = sum n:Nat . rFromChan(n) . B;

 Chan(buff:List(Nat)) =
  (buff == [])
   ->   ((rToChan(0) . Chan([0]))
        +
        (rToChan(1) . Chan([1])))
   <> ((buff == [0])
        -> (sFromChan(0) . Chan([]))
        <> ( (buff == [1]) -> (sFromChan(1) . Chan([])) <> errorAction)
   );

init
 block({sToChan, rToChan, sFromChan, rFromChan },
  comm({sToChan | rToChan -> cToChan, sFromChan|rFromChan->cFromChan },
   A(0) || B || Chan([])
  )
 )
;
```

The core of any mCRL2 model is the `proc` section, where we specify what actions each process can take, and `init`, where we specify the actual processes that our model initially consists of, here `A(0) || B || Chan([])`. We specify that these processes are running in parallel using the parallel processes operator `"||"`. Here, we model 2 nodes `A` and `B`, as well as a message channel `Chan` between them. In this model, we alternate between sending 0

and 1 from `A` to `B`, via the message channel `Chan` process.

In the `act` section, we specify the different actions that may be taken in the model, as well as the parameters that each action takes. We have a set of actions for a message going from `A` to the channel `s/r/cToChan`, and another set for a message travelling from the channel to `B` `s/r/cFromChan`. Both sets take a natural number, which is our message. Finally, we also have an action `errorAction`, which we use to indicate an unexpected state.

To synchronise actions across multiple parallel processes, we specify *communicating* actions using the `comm` operator. The individual actions are specified across different processes, but now the *communicating* action will take both actions in parallel, across the different processes. Using the `block` operator, we can block individual actions, thereby ensuring that *only* the communicating actions are available in the model. In our example, the model allows for the (communicating) `cToChan` action where `A` sends `n` and simultaneously `Chan` receives this `n`, but not for the (individual, blocked) actions `sToChan` and `rToChan`.

We use the composition operator `"."` to put multiple actions in sequence. In process `A`, we specify that `A` takes an action `sToChan` followed by some future action in `A`, where `A` now has a different value for `n`.

In mCRL2, we can provide the option to the model to non-deterministically choose between actions with the choice operator `"+"`. If we want to provide such a choice for different values of a data type, we can use the `"sum"` operator. In process `B`, we specify that `B` can do `rFromChan` for any natural number `n`.

Finally, we can define some action or actions that may only be performed if a certain condition is true, using the `"->"` operator. In `Chan`, if the channel has no messages currently (`buff == []`), we can then choose to receive 0 or 1 with `rToChan`, after which we store this number in our `buff`. We can use the `"<>"` operator to specify an *else* case with the `"->"` operator. In `Chan`, if the channel *does* currently have a message (`buff != []`), then we `Chan` looks to see if `buff == [0]` or `buff == [1]`. If so, we perform `sFromChan` with this stored number, and we empty `buff`. If `buff` is *not* `[]`, `[0]` or `[1]`, then `Chan` can perform `errorAction`. We will later check that `errorAction` can never be taken.

In mCRL2, we can also specify functions in our property. If we wanted to specify alternating between 0 and 1 not in the process declaration for `A`, we could instead do the following:

```
map alternateN: Int -> Int;
eqn
 alternateN(0) = 1;
```

```
 alternateN(1) = 0;
```

and then call declare the process as `A(n:Nat) = sToChan(n) . A( alternateN(n) );`

Example 2

Next, we expand our example, by allowing the channel in between `A` and `B` to store multiple numbers, say some constant `MAX_M`, and by allowing `B` to receive messages out of order. We show this improved example below, where we leave out the unchanged `init` section:

```
map alternateN: Nat -> Nat;
eqn
 alternateN(0) = 1;
 alternateN(1) = 0;

map
 remove : List(Nat) # Nat  -> List(Nat);
 add    : List(Nat) # Nat  -> List(Nat);
var
 ch : List(Nat);
 x, m: Nat;
eqn
 remove([], m) = [];
 (x == m) -> remove(x |> ch, m) = ch;
 (x != m) -> remove(x |> ch, m) = x |> remove(ch, m);

 % ensure that resulting list is sorted:
 add([], m) = [m];
 (m <= x) -> add(x |> ch, m) = m |> x |> ch;
 (m > x) -> add(x |> ch, m) = x |> add(ch, m);

map MAX_M : Nat;
eqn MAX_M = 3;  % Specifies the size of buff.

act
 sToChan, rToChan, cToChan: Nat;
 sFromChan, rFromChan, cFromChan: Nat;
 reportBool : Bool;

proc
```

```
A(n:Nat) =
 sToChan(n) . A( alternateN(n) );

B = sum n:Nat . (rFromChan(n) . reportBool(n == 0) . B);

Chan(buff:List(Nat)) = sum n:Nat . (
 ( (n in buff) -> sFromChan(n) . Chan( remove(buff, n) ) )
 +
 ( ( # buff < MAX_M ) -> rToChan(n) . Chan( add(buff, n) ) )
);

init ... ;
```

mCRL2 provides a number of functions for prepending an item to a list `"|>"`, checking to see if an element is in the list `"in"`, and checking the size of a list `"#"` (1). It is good practice to sort buffers, to ensure that different orderings of messages arriving at buffers does not create more states unnecessarily (24). Here, we allow `B` to read any message currently in the buffer, and as such the order of arrivals is irrelevant. We ensure `buff` is sorted with the `add` and `remove` functions, where the `remove` function removes the *first* instance of this value `n` in the list. These functions are specified in the usual functional programming style: `x` is the head of the current (sub)list, and `m` is the element we are adding/removing.

We now provide a choice in the `Chan` process. If there is some value `n` in the `buff` list, we have the option to send this `n` on to `B`. Alternatively, if the buffer is not full (`# buff < MAX_M`), then we can also receive a new `n` from `A`. We can specify in `eqn MAX_M = ...;` the maximum number of messages that the channel is allowed to store at any time.

2.2.3.2 Regular modal μ-calculus

Once we have a model specification, we can then start checking our verification efforts. If we have specified a finite model, then we can generate the finite state space using mCRL2's `lps2lts` tool (4). However, manually examining the model becomes impractical for models consisting of thousands or millions of states. Instead, we can use mCRL2's tools for model checking formulas automatically.

We specify the grammar for formulas in mCRL2, based in part on the grammar in (1),

where we leave out parts of the grammar not used in this thesis:

$$\alpha ::= a \mid ...$$
$$af ::= \alpha \mid true \mid false \mid !af \mid af \text{ “\&\&” } af \mid af \text{ “||” } af \mid ...$$
$$R ::= af \mid R.R \mid R + R \mid R* \mid ...$$
$$\phi ::= true \mid false \mid \text{val } (b) \mid \phi \wedge \phi \mid \phi \vee \phi \mid$$
$$[R]\phi \mid \langle R \rangle \phi \mid mu\ X.\phi \mid nu\ X.\phi \mid X \mid forall\ d : D.\phi \mid exists\ d : D.\phi \mid ...$$

where a is some action in the model, b is a closed boolean formula, and d is a element of type D.

Hennessy-Milner logic

mCRL2 allows properties to be formulated in regular modal μ-calculus, an extension of Hennessy-Milner modal logic (30). Hennessy-Milner logic expands on propositional logic with box ([...]) and diamond ($\langle ... \rangle$) operators. A formula $\langle a \rangle \phi$ holds in a state if, by performing action a, we can reach a state where ϕ holds.[1] Similarly, $[a]\phi$ holds if ϕ holds in *all* states that are reachable from the current state with an action a.[2]

Regular formulas

This is then expanded upon in (44). Instead of a single action a, we can now have a series of actions R within the box and diamond operators: we now have the composition `"."` and choice `"+"` operators, as well as the Kleene star operator `"*"`. We will discuss these operators with some example formulas, using the specifications provided above. [3]

Using *true* as an action

We start with a simple, and commonly used, example: we want to specify that our example models do not have **deadlocks**. In other words, in any state, there must always be at least 1 action possible. We can use *true* to indicate *any* action, and similarly *false* to indicate *no* action. We can specify “there must be at least 1 action possible” with $\langle true \rangle true$. For this to hold in a given state, it must be possible to take *some* action here.

We want to check that this formula holds in *any* state in the model. Using $[true*]$, we allow for any set of actions (including taking no action), and thus we include any (reachable) state. Thus, we can check that the entire model has no deadlocks using the formula $[true*]\langle true \rangle true$.

[1] Note that, if the action a cannot be performed in this state, then the formula $\langle a \rangle \phi$ does *not* hold.
[2] If the action a cannot be performed in this state, then the formula $[a]\phi$ always holds.
[3] Formulas using these operators can be converted into equivalent, non-regular, μ-calculus formulas (25).

Using *false* to show that a trace cannot occur

In our specification for example 1, we expect that `buff` is either going to be empty, or else equal either `[0]` or `[1]`. Part of this expectation is that `A` is (only) going to send 0s and 1s. If this is not the case, and `buff` for example contains a number other than 0 or 1, then the `Chan` process can perform `errorAction`. We can check that this action is never taken in the model, meaning that `buff` only has the expected values. To this end, we can use the formula $[true * .errorAction]false$. As before, *true** corresponds to any set of action. Using the composition operator `"."`, we can read $[true * .errorAction]$ as "any set of actions *ending with errorAction*." The formula can only hold if there are *no* states reachable with the trace `true* . errorAction`. As such, by checking the formula and finding that it holds, we know that `errorAction` can never be performed in the model, and as such we know that `buff` can only take the expected values.

In the specification for example 2, we are less restrictive in `Chan`, and `buff` is allowed to store any number(s). However, the function `alternateN` is only specified for 0 and 1. If `A` would have a different value for `n` at any time, then mCRL2 would raise an error while trying to generate the state space, as `alternateN(n)` would not be specified for this value.

Using the + operator to reach more states

Next, we show an example which makes use of the `"+"` operator. For this, we have the formula $[true*.(cToChan(0)+cToChan(1))]\ \langle true*.(cFromChan(0)+cFromChan(1))\rangle true$. This formula says that, after every `cToChan` action ($[true * .toChan(..)]$), we should eventually get some `cFromChan` action ($\langle true * cFromChan(..)\rangle$).

The `"+"` operator inside the box here means we take any set of actions ending in *either* `cToChan(0)` or `cToChan(1)`. This results in a larger set of states than for example $[true * .cToChan(0)]$.

The `"+"` operator inside the diamond then means that we can *choose* to eventually do *either* `cFromChan(0)` or `cFromChan(1)`.

Thus, the formula holds if, for any state we can reach by performing `cToChan`, we can reach some state where we can perform `cFromChan`.

Quantifiers

Formulas in mCRL2 may include data, such as the 0s and 1s in the previous example. However, mCRL2 also allows for the use of quantifiers over data. In this thesis, we will limit the use of quantifiers to *finite* data types.

As an example, take the formula $forall\ b : Bool.\langle true * .reportBool(b)\rangle true$. Here, we specify that both `reportBool(true)` and `reportBool(false)` should be possible actions *at some point* in the model.

Fixed point operators

Finally, to truly have μ-calculus, mCRL2 allows for the use of the minimal ($\mu X.\phi$) and maximal ($\nu X.\phi$) fixed point operators. We will focus on the μ operator (written in mCRL2 as mu), as the ν operator is not used further in this thesis.

As an example, we take a formula of the form $mu\ X.[!af]X \wedge \langle true\rangle true$.

With the minimal fixpoint $mu\ X$, mCRL2 aims to construct the *smallest* set of states X such that, for any state $s \in X$: 1) s can perform some action ($\langle true\rangle true$); and 2) for any state s' reachable with actions $!af$, $s' \in X$.

To construct this, we start off with $X = \emptyset$. We then add states s to X where the following holds: the state s has at least one transition ($\langle true\rangle true$), and any state s' reachable from s must be reachable with actions in $!af$. As an example, if $!af$ =!`a` for some action `a`, then in the first iteration we add states s if they *only* have transitions with action `a`. In the second iteration, we then add states that using any action reach a state already in X. (etc.) We keep iterating until we reach an iteration where the new set X' equals the set X. This is then our minimal fixpoint.

mu X **example**

Using the diamond operator, we can show that over *some* path, we eventually take an action. Similarly, the box operator allows us to show a property that holds for all paths *where we take some action.* However, neither operator allows us to show that all paths are guaranteed to eventually take some action. As an example, the property above where we demonstrate the choice operator `"+"` shows that, for all paths where `cToChan` is performed, we *can* afterwards perform a `cFromChan`.

However, we can use μ-calculus to be more specific. Take the following formula:
$[true * .cToChan(1)]\ mu\ X.[!cFromChan(1)]X\ \&\&\ \langle true\rangle true$.

We start from any state s where the last action was `cToChan(1)`. s must then be in the set of states X. As we explained before, from any state in X, including these states s, we must be able to perform some action, and if we take an action that is not `cFromChan(1)`, then the resulting state must also be in X. As such, the formula holding implies that we can guarantee for any state s where the last action was `cToChan(1)`: we are guaranteed to eventually do `cFromChan(1)`.

In other words, if a formula of the form $mu\ X.[!a]X \wedge \langle true \rangle true$ holds, then we are guaranteed to take the action a in *any* path from the starting state.

As a final note: formulas of the form $mu\ X.[!a]X \wedge \langle true \rangle true$ generally do *not* hold when the model has transitions from states to the same state. As an example: if for every state s in the model there is at least 1 action $\neq a$ returning to the same state s, then given that s was not originally in $X = \emptyset$, s in turn will continue to not be a member of X. We can only construct a non-empty set X, if there is at least one state where *any* outgoing action is a.

2.2.3.3 Bisimilarity

In order to reduce the size of the files that we work with, we reduce the size of our models using mCRL2's *ltsconvert* tool (4). (See Section 6.1.) Specifically, we create an *equivalent* model that preserves strong bisimilarity, usually referred to as just bisimilarity (25).

Two models M and M_b are bisimilar if there is a bisimulation relation R such that sRs_b holds, where s and s_b are the initial states in M and M_b respectively. For all pair of nodes x and y where xRy holds, the following must hold for all actions a:

1. If x can take action a and reach a state x', then there must exist a state y' which y can reach taking action a such that $x'Ry'$.

2. If y can take action a and reach a state y', then there must exist a state x' which x can reach taking action a such that $x'Ry'$;

3. The system modelled in M has terminated in state x (so we can take no actions in state x) if and only if the system modelled in M_b has also terminated in state y .

In order to show that a μ-calculus formula holds in M, it suffices to show that it holds in M_b.

As an example, take the formula $[true * .errorAction]false$ again. Say this property does *not* hold in M. Then there is some state x, reachable from the initial state s, such that we can reach x' with $errorAction$. We know that sRs_b holds. Then, by definition of bisimilarity, there must be some state y reachable from the initial state s_b such that xRy holds. Then, there must also exist a state y' reachable from y with the action $errorAction$. Therefore, the property also does not hold in M_b.

Following the same logic, if the property does not hold in M_b, then also it cannot hold in M.

3

Related Work

3.1 Other Termination Detection algorithms

The work in this paper is directly based on (37). Here, the authors create a fault-tolerant version of Safra's algorithm, originally specified in (15, 18) and optimised in (12).

For an overview of other termination detection algorithms, we refer to (19, 46).

We specifically highlight (59), as we will discuss it further below. This is a fault-tolerant termination detection algorithm based on *weight-throwing*, inspired by (34, 47). The main idea behind such weight-throwing algorithms is that we attach some 'weight' to each message that a node sends out, and that passive nodes send the weight they received back. When all nodes are passive, and when each node either has all of its weight back or has sent all of their weight back to some other node, we can then detect termination.

3.2 Model checking

We found only limited examples of work model checking termination detection algorithms. We discuss two such examples in Section 3.2.1. However, there has been considerably more work put into the formal verification of mutual exclusion and consensus algorithms. We discuss a number of such papers in Sections 3.2.2 and 3.2.3.

A number of different tools exist for model checking distributed algorithms. Some popular examples include Promela/SPIN (31), TLA$^+$ (40), mCRL2 (4), and CADP (21). In (43), the authors use CADP to perform *both* model checking and performance evaluation.

While most of the above tools are used to model check abstract specifications of algorithms or systems, tools such as (63) can model check actual implementations.

3.2.1 Termination detection algorithms

Similar to the work in this thesis, (39) also creates a formal specification of Safra's algorithm, and performs verification on this specification. The authors do this in order to demonstrate a number of features of the TLA$^+$ Toolbox (40). The main difference between this paper and our work is that (39) specifies and verifies properties of the *original* version of Safra's algorithm, as set out in (15, 18). We instead work on the *optimised* version of Safra's algorithm (12), and particularly on the *fault-tolerant* version set out in (37). When working with TLA+'s explicit-state model checker `TLC`, the authors limit the number of messages sent (as in our specification), but do *not* need to limit the number of iterations that the token takes. This is a result of the difference between the original version of Safra's algorithm (15, 18), and the optimised version (12) which uses an explicit sequence counter. (See Section 4.1.)

Furthermore, we define some of our main properties slightly differently. In (39), TLA$^+$ allows for a global overview of the system. As such, the predicate `terminated` is instantly *true* when the basic algorithm has terminated, and this is used in formulas to check properties. In our approach, we primarily observe the state through actions that have been or can be taken. Instead, we have an action that allows us to approximate when the basic algorithm has terminated. We then effectively combine (39)'s $Safe$ and *Quiescence* properties into a single $Safety$ property. (See Chapter 4.)

In (28), the authors create formal specifications using Uppaal (2) for both the fault-sensitive and fault-tolerant termination detection algorithms using weight-throwing, as presented in (59). The `Termination` process in these specifications appears to be specified similar to how we specify the `Node` process. In (28), the author finds that the algorithm does not satisfy the liveness property, as they find a trace of the algorithm where termination cannot be detected. Furthermore, the author shows that one of the invariants provided by (59) are not guaranteed.

3.2.2 Consensus algorithms

Consensus algorithms are, as the name might suggest, used to get agreement in a distributed system. This could involve the system deciding on a boolean value ('Is the system taking the True or the False branch here?'), or indeed whether an algorithm has terminated.

Raft (49) and Paxos (42) are generally considered to be the two most popular distributed consensus algorithms in use (33). In (3), the authors use mCRL2 (4) to model check

properties of Raft, while (11, 60) using different approaches to verify properties of Paxos.

3.2.3 Mutual exclusion algorithms

The mutual exclusion problem appears to originally have been formulated in the 1960s in papers such as (13). In (14), Dijkstra describes the problem as follows: we have N nodes, each with a "critical section". At any time, no more than one computer may access its "critical section". These computers can communicate with each another using shared storage, such as a shared register. [1] Reads or writes are atomic; if two nodes simultaneously try to access the same register, these actions will occur in sequence, but in an unknown order. Any mutual exclusion algorithm needs to ensure that no more than 1 node can access the critical section at any time. Any such algorithm should also ensure that every node eventually gets to access its critical section (known as 'starvation-freeness') (38).

A number of different mutual exclusion algorithms have been formally analysed, either resulting in formal verification of their correctness, or resulting in issues being identified in the algorithms.

In (26), the authors use mCRL2 (4) to first create a model of Peterson's algorithm for $N \geq 2$ nodes (53) and specify the expected properties. This algorithm sets processes as a set of leaves in a binary tree, with a shared register assigned to each internal node in the tree. Using this shared register, the algorithm decides which process can 'move up' in the tree. A processor at the top of the tree then becomes privileged. The authors found that in their models for $N = 3$, 4, and 5, the algorithm was not starvation-free, violating one of the expected properties of a mutual exclusion algorithm. Using the mCRL2 toolset, the authors generated a counterexample to show this violation. They then created an altered version of the algorithm, "inspired by [this] counterexample" (26), and showed that this *did* satisfy starvation-freeness for $N = 3$ and 4, as well as provided a paper proof for this property holding for $N \geq 4$.

In fact, Peterson's algorithm for $N = 2$ nodes (52, 57) [2] and the more general version for $N \geq 2$ nodes (53) have been formally verified and/or analysed in numerous papers (8, 26, 36, 43, 61), each using a different tool.

In (56), the author first formally verifies the (fault-sensitive) Ricart-Agrawala mutual exclusion algorithm (54) [3] in Coq (64). The author then presents a fault-tolerant version

[1] Note that this is contrary to the message-passing paradigm we use in the rest of this thesis.

[2] According to (36): "it's easy to prove that [these two versions of Peterson's algorithm for two processes] are equivalent."

[3] The original Ricart-Agrawala mutual exclusion algorithm (54) had been verified before in, in (55).

of the algorithm, and creates a TLA$^+$ specification of it to model check (40).

4

Overview

In this section, we will give a high level overview of the approach we have taken to create our models for the fault-sensitive (12) and fault-tolerant (37) versions of our algorithm.

We provide an upper bound to the number of times that the token can go around the ring for our two algorithms, by means of a paper proof for the fault-sensitive algorithm, and some intuition towards a similar proof for the fault-tolerant algorithm. This shows that the limitations we set on our models indeed result in a *finite* state space.

We will then describe, in words, a series of properties that we wish to check on our models.

In Chapter 5, we will then discuss the actual specifications we have drawn up, and we will show the formulas that we use to model check our properties.

4.1 High-level approach to the model specifications

Similar to the approach taken in e.g. (28, 39), the nodes are the centre point of our model, from which most actions originate.

We have N parallel nodes, and our specifications allow each node to non-deterministically choose between a set of actions: a node i can receive incoming basic messages from another node j and, provided that the node is active, i may also send such basic messages to a node j. An active node may choose to become passive, and similarly a node may become *done* if it has not previously done so. (*done* will be defined below) A node may also receive an incoming token. If the node is currently active, i will buffer this token until it becomes passive. Once i becomes passive with a token stored, or if i is already passive when it receives the token, it will then proceed to handle the token in accordance with the control

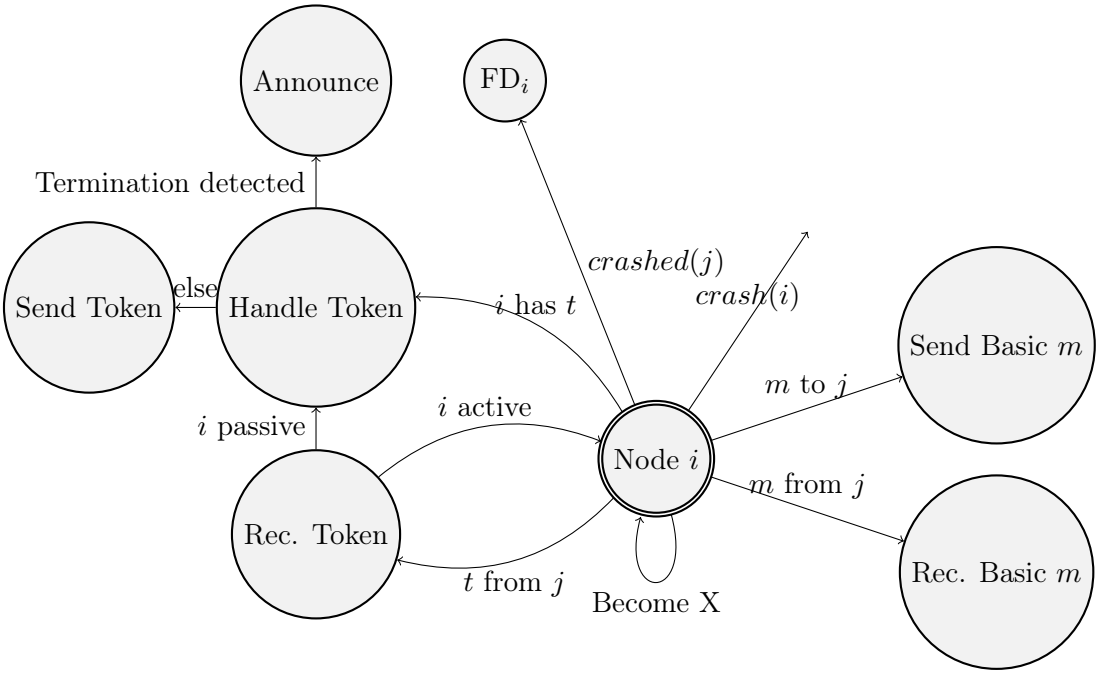

Figure 4.1: The different procedures that we can use, with Node as the starting point. After executing these procedures (except for $crash(i)$), we return to Node. Handle Token executes the part of the ReceiveToken$_i$ procedure 10 *after* the $wait(passive_i)$ call.

algorithm. Once the node has completed its current procedure,[1] it then returns to the main section of the Node specification, from which the next action can be chosen. See Figure 4.1.

We aim to abstract away the basic algorithm, as the focus of our model is on the *control* algorithm. To achieve this, we provide each node in the system the (non-deterministic) choice between each of the actions detailed above. The resulting state space then effectively contains any execution of a basic algorithm, within our set limitations (see below), from the perspective of our control algorithm.

As we wish to model asynchronous communication, we use separate objects to buffer messages (basic and control) between pairs of nodes.[2]

[1] Remember that we assume that procedures are *not* interrupted, except for $wait(passive_i)$ calls, see Section 2.1.3.1.

[2] Note that our resulting state space also includes executions where we effectively have *synchronous* communication: namely any trace where a message being sent to a buffer is immediately followed up by the message being received from the buffer.

4.1.1 Failure detector

As discussed in Section 2.1.3.3, the fault-tolerant algorithm requires a *perfect* failure detector. We will model this by splitting the failure detector: we have one single global failure detector process, which keeps track of the set of nodes that have crashed, and each node i has a local failure detector procedure, executing FailureDetector$_i$ 8. In this local procedure, node i is informed by the global failure detector that some other node j has crashed.[1]

We then create a number of places in the model where each node can crash, see Section 4.1.4.

4.1.2 Limitations on the model

As with the model specification in (39), we need to place limitations on the models to avoid an infinite state space. Similar to (39), we limit the number of basic messages that can be sent by all nodes combined to some constant M.[2] (See Section 5.1.4 for details.)

However, we require an additional limitation. In our versions of Safra's algorithm ((12) & (37)), we use sequence counters seq_i and seq_t. The token being at node 0 with $seq_i = 0$ and the token being at node 0 with $seq_i = 1$ will be two separate states in our model, while in (39) this could be the same state. While there is some basic message in a buffer between nodes, the basic algorithm has not yet terminated. If all nodes are passive, and this message does not arrive, then the nodes can keep passing the token around the ring, increasing $seq_i/_t$ and thereby creating new states in our state space.

We explicitly want to *include* the sequence counters in our model, as they form an intrinsic part of both versions of Safra's algorithm that we wish to check, for checking if we need to update $black_i$ and for dismissing 'old' tokens in the tolerant algorithm. Further, assumptions about fairness would also likely not provide a solution here. As we show in Section 6.2, even a few iterations of the token can lead to very large state spaces.

In order to still achieve a finite state space, we limit the number of times the token can go around the ring to some constant S.[3] After the token has done S many iterations, we

[1] An alternative approach is taken in (28): the crashed node(s) continue to operate, but a crashed node is restricted to only informing other nodes that this node has crashed.

[2] In (39), the authors set $count_i \leq C$ for some constant C (in their example, $C = 3$). Among other things, this means that each node can send at least C messages. Each node that is added to the model thus allows for C more messages to be sent. We choose not to do that here, instead opting for our single global limit M. The state spaces of our models, even for small M, are already quite large and grow quickly as N increases, see Section 6.2.

[3] We refer to a similar S in our proofs in Section 4.2.

first require the basic algorithm to be *done*. We define *done* as follows:

Definition 4.1.1 (*done*)**.**

- The basic algorithm is *done* when all nodes and message buffers have become *done*.
- A node becomes *done* when it does the `becomeDone` action, or when it crashes. Once a node does the `becomeDone` action, it may no longer send any *basic* messages. The node *can* still receive basic messages, crash, and act as work as usual on the control algorithm.
- A message buffer may become *done* when it is empty, when the receiving node has crashed or when the receiving node knows that the sender has crashed.[1] The message buffer may not receive basic messages once it becomes *done*. In practice, this means we only allow message buffers to become *done* after *all* nodes are *done*.

In practice, we maintain a separate process that keeps track of which nodes/message buffers are *done*, and determines which node/buffer may next become *done*.

Once the basic algorithm is *done*, we know that there is only a finite number of iterations that the token can make around the ring. (See Section 4.2.)

As an added benefit, we can use this definition as an approximation of the basic algorithm terminating: when the basic algorithm is *done* and all nodes are passive, we know that the basic algorithm has also terminated: there are no incoming basic message that will make nodes active.[2] Contrary to for example (39), we cannot (easily) determine with our approach when the underlying basic algorithm has terminated.

4.1.3 Blocking actions

When creating a model, it is easy to create a specification where we have deadlocks in the model. Specifically, it is easy to create a scenario where, given a list of choices, we take a certain action, and then later down this chosen 'path' we encounter an action that we cannot take. In the main node procedure, we set out a list of different options for this node i, such as sending or receiving a basic message. Once i takes the first action on a specific path, it can no longer choose from the previous list of choices, until i finishes this path and returns to the main node procedure. If i could lock itself into the path for receiving a

[1] Note that this definition resembles the definition of termination provided in Section 2.1.3.3.

[2] However, note that the basic algorithm may have terminated *before* we determine that the basic algorithm is *done*, and as such the control algorithm may already have detected termination before we determine that the basic algorithm is *done*. See property **P5** in Section 4.3.

basic message, but no basic message would ever arrive for i to receive, then clearly i would end up in a deadlock on this path.

We aim to avoid deadlocks in the model *before detecting termination*. To achieve this, we set out to put the actions that might block a certain path at the *start* of that path. As an example, we specify that the first action node i takes on the path to sending a basic message, is to get permission to send this message, where the permission ensures we have not exceeded the limit M. If we had specified a different order of actions, then i may for example first send out the message and *then* ask for permission. If no permission is granted (if the network was already at the limit M), then node i deadlocks. In our models, nodes need to explicitly perform an action in order for the failure detector to consider them crashed. As such, a deadlocked node means the liveness property will be violated in our model, despite such a deadlock not occurring in an actual implementation.

4.1.4 Adding crashes to the specification

In order to faithfully model the fault-tolerant algorithm, nodes should be able to crash in the middle of operations. However, due to the state space explosion problem, we want to limit the number of places where crashes can occur. Specifically, we set out to avoid any options to crash that are roughly equivalent to another option we provide for crashes.

As a starting point, one of the choices we can make from the main node procedure is for the node to crash.

Then, we split all possible actions that a node can take into 3 groups: actions in our model that do not occur in a real implementation; internal actions; and external actions, which affect the larger network. Examples of internal actions include updating local variables or a node becoming active/passive, while external actions include sending/receiving messages, Announcing, or a node crashing.[1] Actions in our models that would not occur in a real implementation includes actions related to *done*, as well as actions related to a node ensuring that we have not hit our limit of M basic messages.

We only wish to add the option for a node to crash in places where we do two consecutive external actions, without first returning to the main node procedure. For most external actions, we do exactly 1 before returning to the main node procedure. As an example, take the process of sending a basic message: the only external action required is sending a basic message, after which we return to the main node procedure. Similarly, a crash *before*

[1] In mCRL2, we often hide such internal actions, where they get replaced with the hidden action(s) τ. As we will these internal actions for checking our properties, we choose not to hide these in our models.

some internal action would be effectively equivalent, from the perspective of the network as a whole, as only allowing this crash *after* this internal action.

Turning to Figure 4.1, there is only one pair of consecutive external actions: when node i receives the token, if i is passive, we move directly to handling the token where we will either send off the token or Announce, but *without* returning to the main node procedure first. Where i may go directly from receiving the token to handling the token, we provide i the option to instead crash.

4.2 Formal analysis of termination of Safra's algorithm

Further on in this work, we model check, among other properties, that the control algorithm always terminates, provided that the basic algorithm terminates. (The Liveness property.) In this section, we provide a paper proof showing that the Liveness property holds in the fault-sensitive algorithm, as well as a sketch for the same proof for the fault-tolerant algorithm. This shows that, with our limitation S set on the models, we get *finite* models.

$black_i$ stays within the range of IDs

We start by briefly discussing the following lemma, showing that $black_i$ and $black_t$ always refer to IDs that are actually in our system:

Lemma 1. *In the fault-sensitive algorithm (12): for all nodes i and tokens t:* $0 \leq black_i, black_t < N$.

Whenever a value is assigned to $black_i$ or $black_t$, we assign either 0 or $N-1$ (Initialisation$_i$ 1), we assign $black_i$ or $black_t$ itself (e.g. ReceiveToken$_i$ 4), or we assign another node j (e.g. ReceiveBasicMessage$_i$ 3). 0 and $N-1$ are initially within our bounds, and we assume that, for all IDs j, $0 \leq j < N$. By induction, we thus see that $black_i$ and $black_t$ stay within our expected bounds.[1]

Lower bound on $black_i$ when the basic algorithm terminates

We discuss a further lemma, which we will use to show termination is guaranteed for the fault-sensitive algorithm (12):

Lemma 2. *In the fault-sensitive algorithm (12): when the basic algorithm has terminated and the token is at node 0, $black_i \geq i$ for all nodes i.*

[1] While we do not explicitly write a property to model check this in Section 4.3, mCRL2 would likely raise an error while generating our state space when providing an out-of-bounds value to (e.g.) our *NatToID* function, see Section 5.1.3.

Proof.
Let the token have completed S iterations for some $S \geq 0$ when the basic algorithm has terminated. As we increment seq_i each time the token passes the node i, $seq_i = S$ for all nodes i.

$black_i$ is set in three places:

First, in Initialisation$_i$ 1, where $black_i = i$ for all nodes i.

Second, in ReceiveToken$_i$ 4. When the token is sent off to the next node, $black_i$ is set to i. If we detect termination, then $black_i = i$. (Line 3.)

Finally, in ReceiveBasicMessage$_i$ 3, where $black_i$ might be set to some ID j. Note that when the token left node i for iteration S, we set $black_i$ to i. We thus need to show that, after i passed on the token in iteration S, $black_i$ remains $\geq i$.

$black_i$ is only updated in ReceiveBasicMessage$_i$ 3 if, for some incoming message m from a node $j \neq i$, either $seq_m = seq_i + 1$ or $seq_m = seq_i \wedge j > i$ is true.

We assumed that the basic algorithm had terminated when $seq_k = S$ for all nodes k (including i, j). As such, there cannot be a basic message sent with $seq_m = S+1 = seq_i+1$, so this first case cannot occur.

Now let us take the case where $seq_m = seq_i \wedge j > i$ holds. Then, we assign to $black_i$ $furthest_i(black_i, j)$. $furthest_i$ will return either $black_i$ or j. By induction, $black_i \geq i$, and given is that $j > i$, which implies $j \geq i$.

Thus, we assign a value $\geq i$ to $black_i$, so $black_i \geq i$ remains true. □

While we do not believe that this affects the larger correctness argument for the two theorems below, note that such a bound would not be applicable to the fault-tolerant algorithm, due to the assignment to $black_i$ in NewSuccessor$_i$. As an example: let node $N-2$ have $black_{N-2} = N-1$ (valid under Lemma 2's bounds) with all nodes being non-crashed. Now let $N-1$ crash. If $N-2$ detects this crash, NewSuccessor$_{N-2}$ will then set $black_{N-2} = 0$, resulting in $black_{N-2} \not\geq N-2$, thus violating Lemma 2's bound.

Liveness proof for the fault-sensitive algorithm

We show that, once the basic algorithm terminates, termination is detected shortly thereafter in the fault-sensitive algorithm (12):

Theorem 3. *Once the basic algorithm terminates, the token passes node 0 no more than once before detecting termination.*

Proof.
Without loss of generality, assume the token is at node 0, and node 0 has not yet executed

ReceiveToken$_0$.[1]

By Lemma 2, we know that $black_i \geq i$ for all nodes i at this point. Specifically, we then know that $black_{N-1} = N - 1$ using Lemma 1.

The basic algorithm's termination implies that all basic messages have arrived. As such, starting from node 0, no later than Node $N - 1$ do we find that $count_t = 0$, as $count_t$ will account for every basic message being sent ($+1$) and received (-1).

Starting from node 0, with $black_i \geq i$, we know that $black_t \leq N - 1$ during this $S + 1$'th iteration. No later than node $N - 1$ do we find a node where $black_i = i$.

In other words, no later than node $N - 1$ do we find that $count_t = 0 \ \wedge \ black_i = i$, allowing us to detect termination in ReceiveToken$_i$ 3. □

We show that this property holds in our models, by checking property **P11**, which tells us that there is an upper limit on seq_i in our models, combined with **P0**, which tells us that we always eventually Announce. (See Section 4.3.)

Liveness proof sketch for the fault-tolerant algorithm

As with the fault-sensitive algorithm, we can specify an upper bound on the number of iterations before we detect termination in the fault-*tolerant* algorithm (37):

Theorem 4. *Once the basic algorithm terminates, the token completes $\leq N - 2$ iterations before the fault-tolerant algorithm detects termination, where $N \geq 2$ is the number of nodes in the ring.*

We will not prove this holds, however we provide some intuition here towards its validity, and show that it holds in our models in by checking property **P12**, showing an upper bound on seq_i, combined again with **P0**. (See Section 4.3.)

In the case where there are no crashes, the proof largely follows the proof of Theorem 3, as we quickly detect termination.

Each time some node crashes, we can delay termination detection by one iteration: In FailureDetector$_i$ 8, if $next_i$ crashes, i sends out the (possibly lost) token[2] with $black_t = i$, ensuring that the token must at least return to i before we can detect termination, and with $next_i$ added to $Crashed_t$.

Alternatively, if some node i detects j's crash, and $j \notin Crashed_t$, then in HandleToken$_i$, we can set $black_t = i$ (Line 21).

[1] If the token is not yet at node 0, we can simply start by carrying out the algorithm until the token arrives at node 0.

[2] While we do in effect *duplicate* tokens this way, due to the $seq_t = seq_i + 1$ check in HandleToken$_i$, the algorithm ensures that no more than 1 token is actually being passed around the network.

In either case, for each crash, we can delay termination detection by approximately one iteration.

The exception to this is when the penultimate node crashes, leaving only 1 non-crashed node i. In this instance, i does not send out a new token, but instead detects termination in NewSuccessor$_i$ 9.

Each node can crash exactly once, and we assume that at least 1 node will not crash. As such, after (at most) $N-2$ iterations, one for each crash excluding the $N-1$'th crash, we should always detect termination in some node.

4.3 Properties to verify

Once we have a model, we can then check properties on it. Here, we set out a number of properties in words that we wish to check. In Section 5.3, we will then specify these formulas in mCRL2, using actions specified in our models.

We will not specify any properties that require us to quantify over tokens, given the complexity of the token objects, especially for the fault-tolerant algorithm.

To start, we set out a series of simple properties to check that our model works as expected:[1]

- **P0**: The model has no deadlocks before Announcing. In other words, all traces in the model eventually reach Announce.
- **P1**: All nodes i can Announce.
- **P2**: All nodes i can crash.
- **P3**: All nodes i can send a basic message to and receive a basic message from all nodes j, where $i \neq j$.
- **P4**: We can reach Announce *with* the network being *done*.
- **P5**: We can reach Announce with*out* the network being *done*.

We check these properties to verify some basic actions or traces are available in the model. As discussed in Section 2.2.3.2, a formula of the form $[true*.a]\phi$, where an action a is never taken, always holds, regardless of ϕ. With these properties, we check that such actions a are actually taken in the model. This then means that properties of the form $[true*.a]\phi$

[1] Specifying and checking these relatively simple properties as we develop the model specifications is also a useful method to debug the model specifications.

actually reaches some *non-empty* set of states, meaning that we will actually check the property in ϕ.

We can then move on to checking the properties on our actual algorithm: First, the two mentioned in (37):

- **Liveness**: If the basic algorithm has terminated, this termination is eventually detected by a (non-crashed) node.
- **Safety**: When termination is detected, the basic algorithm has indeed terminated at some point in the past.

In addition to this, we have some further properties that we want to verify, to confirm smaller properties of the algorithm:

- **P6**: In the fault-tolerant setting: If a node i Announces and then crashes, we always eventually have some other node j that Announces.
  Note that, because we assume that at least one node will not crash, we will eventually have some node that does *not* crash that Announces.
- **P7**: For each message m received by a node i: $seq_m \leq seq_i + 1$.
  The algorithm adjusts $black_i$ when relatively 'new' messages arrive/ However, the algorithm assumes that these 'new' messages are still $\leq seq_i + 1$.
- **P8**: In the fault-tolerant setting: For all tokens t received by a node i: $seq_t \leq seq_i + 1$.
  In other words, a token is never dismissed for being 'too new' in ReceiveToken$_i$ 10. (Line 27.)
- **P9**: A node i is never in $Crashed_i \cup Report_i \cup Crashed_t$.
- **P10**: In the fault-tolerant setting: If some node i does *not* update $count_t^i$ in this iteration of the control algorithm (the condition on line 6 of ReceiveToken$_i$ 10 evaluates to *false*), then at least one of the following will always eventually occur:
  - Node i updates $count_t^i$ in a later iteration. (So the condition on line 10 eventually evaluates to $true$ in node i.)
  - Node i crashes.

  - Node i eventually is the final node remaining, and does *Announce* through line 6 of NewSuccessor$_i$ 9.
    In this case, we do not look at $count_t$ in order to detect termination, and thus it is irrelevant whether $count_t^i$ is correct or not.

- **P11**: In the fault-sensitive setting: seq_i may equal any value in the range $[0, S+1]$, and can*not* be higher than $S+1$.
  We can use this to confirm, within the limitations of our model, that Theorem 3 holds, as termination must then be detected no later than the $S+1$'th iteration, where the system becomes *done* no later than the end of the S'th iteration.

- **P12**: In the fault-tolerant setting: seq_i may equal any value in the range $[0,\ S+1+min(N-2,C)$, and can*not* be higher than $S+1+min(N-2,C)$.
  We can use this to confirm, within the limitations of our model, that Theorem 4 holds. If $C < N-1$, then each crash will add an iteration. If $C = N-1$, then the last crash does *not* add another iteration, as discussed in Section 4.2.

With these properties, we check certain assumptions made by the algorithm, or expectations set by looking at the code. As an example, in ReceiveToken$_i$ 10, we only handle the latest token, namely the token where $seq_t = seq_i + 1$. We expect that, under our assumptions, all tokens received are either going to be *older* tokens ($seq_t < seq_i + 1$), or a token we actually work on ($seq_t = seq_i + 1$). In other words, we expect to not have a situation where a token arrives at i with $seq_t > seq_i + 1$. For similar reasons, we expect that *basic messages* have the same property with their sequence counter $seq_m \leq seq_i + 1$.

In the fault-tolerant situation, we expect that the network can handle crashes, even if the node that crashes had detected termination, or was in the process of carrying out Announce. In our model, if some node i crashes after detecting termination, some other node j should eventually detect termination.

Finally, property **P10** ensures that, with some exceptions, if the algorithm does *not* update $count_t^i$ in this iteration, it is guaranteed to update this later.

Implicit property checking

Some properties are not *explicitly* checked with formulas, but are instead implicitly found to hold as a result of us being able to generate our models. As an example: we do not explicitly check that the while loop in NewSuccessor$_i$ 9 is always terminating. However, if this was *not* the case for our instances, then we would not be able to generate the state spaces for these models (62).

5

Design

5.1 Fault-sensitive model specification

In this section, we will discuss our specification modelling the optimised, fault-*sensitive* version of Safra's algorithm (12).

We will highlight and discuss specific sections of the specification here. See (58) for the full specifications, for each value of N we discuss. We will discuss in detail the model for $N = 2$, $M = 2$, $S = 1$.

5.1.1 Assumptions

We follow the assumptions as set out in Section 2.1.3.1. Specifically, this means we work in a system where individual nodes communicate with asynchronous message passing, and with no shared memory or global clock.

We model the basic algorithm by allowing nodes to send and receive basic messages to other nodes. The model does *not* include any internal actions as part of the basic algorithm.

We model a message channel between **every** pair of nodes i, j, where $i \neq j$. Instances where some message channels are absent are included in the resulting model: these are simply the traces in the model where there are no messages sent through these absent message channels.[1]

(37) specifies that "[p]rocedures are executed without interruption", except that a node can continue to send and receive messages after receiving a token, while the node remains

[1] Note that, as discussed in 2.1.3.1, we *require* communication from i to $(i+1)\%N$ for the fault-sensitive Safra's algorithm, and we similarly require communication between each pair of nodes for the fault-tolerant algorithm.

active.

5.1.2 Constants

For our model, we define a series of constants, including our constants for M, S and N:

```
map %Define & set constants
 MSG_LIMIT_GLOBAL, SEQ_LIMIT  : Nat;
 N_PROCS   : Pos;
 N_MSG_BUFFS : Int;
eqn
 MSG_LIMIT_GLOBAL = 2;   % M
 SEQ_LIMIT = 1;          % S
 N_PROCS   = 2;          % N
 % Each node i has a channel to each other j (!= i).
 % We have *separate* channels i->j & j->i.
 N_MSG_BUFFS = (N_PROCS * (N_PROCS-1));
```

5.1.3 Custom data types

mCRL2 provides the option to add custom data types: to create a completely new data type (e.g. `ID`), to use as a type alias (e.g. `BasicMsg` and `MsgBuff`), or to combine other types together into a larger object (e.g. `Token`). mCRL2 automatically gives such custom data types a total ordering[1], meaning that we can compare elements and thus store them in *ordered* buffers.

ID

We create a custom data type for the IDs. This allows us to easily quantify over IDs in properties. As `N_PROCS` is set to 2, we only create two possible values for our data type in this specification. For models where we have more nodes, we add additional values to the type. As an example, if we specify for $N = 3$, we add `... | Two;`.

```
sort
 ID = struct Zero | One;
```

We then create functions for converting from natural numbers to IDs and vice versa. As with the above, if we add more values to the `ID` data type, then we also add the corresponding values to `IDtoNat` and `NatToID`.

[1] See `https://mcrl2.org/web/user_manual/language_reference/data.html`.

Finally, we create another constant: `BLACK_INIT_ZERO` is the value $N-1$ to which we set $black_t$ initially in Initialise$_0$ 1.

```
map
 IDtoNat : ID -> Nat;
 NatToID : Nat -> ID;
 BLACK_INIT_ZERO : ID;
var
 id : ID;
eqn
 IDtoNat(Zero) = 0;
 IDtoNat(One)  = 1;
 NatToID(0) = Zero;
 NatToID(1) = One;
 BLACK_INIT_ZERO = NatToID((0-1) mod N_PROCS); % = N_PROCS-1.
```

BasicMsg and MsgBuff

We create a pair of type aliases: `BasicMsg` and `MsgBuff`. `BasicMsg` represents the basic messages sent by nodes. In our model, the basic message consists only of a sequence number (seq_m) of type `Nat`. `MsgBuff` in turn is a list that of basic messages, stored in each message buffer

```
sort
 % Our (abstracted) basic messages consist solely of the sequence counter seq_m.
 BasicMsg = Nat;
 MsgBuff  = List(BasicMsg);
```

The advantage of using type aliases, is that once can easily change the underlying type without having to rename each of the variables. As an example: say we wanted to expand the model so that the basic message contains more than just seq_m. Then we could redefine what `BasicMsg` is, and adjust the use of the variable accordingly.[1]

In line with (24), we ensure that `MsgBuff` is an *ordered* list of messages. To achieve this, we specify that the `add` function adds a new message `m` in the correct part of the list. Similarly, we define `remove` to always removes the *first* occurrence of a given basic message.

```
map
 % Define operations
```

[1] In fact, the ID data type was originally a type alias for the natural numbers, which we later adjusted.

```
 remove: MsgBuff # BasicMsg -> MsgBuff;
 add : MsgBuff # BasicMsg -> MsgBuff;
var
 mb : MsgBuff;
 x, m : BasicMsg;
eqn
 % We remove the *first* item in the list equalling m.
 remove([], m) = [];
 (x == m) -> remove(x |> mb, m) = mb;
 (x != m) -> remove(x |> mb, m) = x |> remove(mb, m);

 % make the add operations sorted, to ensure that the buffer remains ordered:
 add([], m) = [m];
 (m <= x) -> add(x |> mb, m) = m |> x |> mb;
 (m > x) -> add(x |> mb, m) = x |> add(mb, m);
```

Token and TokBuff

For the fault-*sensitive* algorithm, we store two values in the token: $count_t$, an integer, and $black_t$, an `ID`. As such, the `Token` data type consists of a pair of these two elements.

```
sort
 Token    = struct pair(fst: Int, snd:ID); % (count_t, black_t)
 TokBuff  = List(Token);
```

Similar to the previous section, the `TokBuff` type is simply a list of tokens. As mCRL2 does not support polymorphism (45), we define `addT` and `removeT` in the same way as the `add`/`remove`.

Similar to object-oriented programming, we create a constructor function `Tok` and getter functions `countT` and `blackT`.

```
map
 Tok : Int # ID -> Token;
 countT : Token -> Int;
 blackT : Token -> ID;
 removeT: TokBuff # Token -> TokBuff;
 addT   : TokBuff # Token -> TokBuff;

 tokPrev, tokNext : ID -> ID;
 EMPTYTOK : Token;
```

```
 newTokFurthest : ID # ID # ID -> ID;
var
 fstV : Int;
 secV : ID;
 tokX, tokT : Token;
 tokB : TokBuff;
 idCurr : ID;
 origID, origBlLocal, origBlackT : ID;
eqn
 % Constructor for a token:
 Tok(fstV, secV) = pair(fstV, secV);
 % Get values from the token:
 countT(pair(fstV, secV)) = fstV;
 blackT(pair(fstV, secV)) = secV;
 ... % we omit removeT and addT's definitions here

 % Given the current id, find the previous/next ID in the ring.
 tokPrev(idCurr) = NatToID((IDtoNat(idCurr)-1) mod N_PROCS);
 tokNext(idCurr) = NatToID((IDtoNat(idCurr)+1) mod N_PROCS);

 EMPTYTOK = pair(0,Zero);

 % For HandleToken: if we do not announce, then we update black_t
 % using the result of  furthest_i(black_i, black_t). As we cannot
 % update black_t within HandleToken, we use this to make HanldeToken simpler.
 newTokFurthest(origID, origBlLocal, origBlackT) =
  furthest(origID, furthest(origID, origBlLocal, origBlackT), tokNext(origID) );
```

We use the functions `tokPrev` and `tokNext` to determine the previous/next node in the ring, so that we can receive the token from the previous node / send the token on to the next node.

We specify a constant `EMPTYTOK`: if we do not have a specific token stored, for example initially in the nodes $i \neq 0$, we store `EMPTY_TOK` here.

In ReceiveToken$_i$ 4, we first update $black_i$ and then, if we do not Announce, use this value to in turn update $black_t$. In order to avoid nesting $furthest_i$ calls in `HandleToken` (see Section 5.1.9), we use `newTokFurthest` instead.

5.1.4 msgCounter

As detailed in Section 4.1, we limit the number of basic messages sent across the entire network to M, or `MSG_LIMIT_GLOBAL` in mCRL2. In order to enforce this limit, we use the `msgCounter` process:

```
msgCounter(n:Nat) = sum i:ID . (
 (n < MSG_LIMIT_GLOBAL) -> (sndPermSend(i) . msgCounter(n+1))
);
```

Initially, `n` is set to 0. Whenever a node `i` wants to send a message, `msgCounter` must grant permission by performing the `sndPermSend(i)` action. After `msgCounter` has granted permission `MSG_LIMIT_GLOBAL` times, no more basic messages can be sent.

5.1.5 Node

We store a list of variables in each node:

```
Node(id:ID, black:ID, msgCount:Int, seq : Nat, active:Bool,
    hasToken:Bool,tok:Token, basicDone:Bool) = ...
```

`black`, `msgCount` and `seq` correspond to $black_i$, $count_i$ and seq_i, where $i =$ `id`. We use the boolean `active` to track whether a node is active or passive, and `basicDone` to track whether a node has informed the `doneChecker` that the node is *done*.

If the node has received a token but has not yet started handling it, for example because the node was active when it received the token (line 1 of ReceiveToken$_i$ 4), then we set `hasToken` to true, and `tok` stores this token.

As discussed in Section 4.1, the `Node` process features a number of different actions, which we can non-deterministically choose between (separated by the choice operator `+`).

If node `id` is active, the following two actions are available to `id`: first, if a node `id` gets permission from `msgCounter`, it can send a basic message to some other node `j`, after which we increment $count_{id}$. Alternatively, node `id` can decide to become passive. If we have the token, we start the `HandleToken` process, else we continue on in `Node`.

```
((active) ->
    (
    ((!basicDone) -> sum j:ID .
        ((j != id) ->
            (rcvPermSend(id) . sendToBuff(seq, id, j) .
                Node(id, black, msgCount+1, seq, active,
```

```
                        hasToken,tok, basicDone)
            )
        )
    )
    + ( BecomePassive(id) .
        (hasToken)
            ->  (HandleToken(id, black, msgCount, seq, tok, basicDone))
            <>  Node(id, black, msgCount, seq, false,
                        hasToken, tok, basicDone)
      )
    )
)
+ ...
```

Next, node `id` can receive an incoming basic message. For any incoming message m with seq_m = `newSeq`, we expect that `newSeq` $\leq$ `seq+1`. If this expectation is *not* met, and we get messages with `newSeq` $>$ `seq+1`, then we use the `reportErrorTooNewMessage` action to indicate this.

If `id` was passive before receiving this basic message, it performs `BecomeActive(id)` to indicate that the node now is active. Per the condition on line 2 of ReceiveBasicMessage$_{id}$ 3, we only update $black_{id}$ if certain conditions are met, using the sequence counter `newSeq`. Finally, we then decrement $count_{id}$ as we have received a basic message.

```
( sum j:ID . (j != id) -> sum newSeq:BasicMsg .
    (recFromBuff(newSeq,j, id) .
        % The node becomess active by receiving a message,
        % if it was passive prior.
        ( (!active) -> BecomeActive(id) <> nodeAlreadyActive(id) ) .
        ((newSeq == seq+1 || (newSeq == seq && j > id))
        % we only update black with furthest_i(black,j) if ^ is true.
        % (Line 3 of RBM)
        -> Node(id, furthest(id, black,j), msgCount-1, seq, true,
            hasToken,tok, basicDone)
        <> reportErrorTooNewMessage(id) .
           Node(id, black, msgCount-1, seq, true, hasToken,tok, basicDone)
        )
    )
)
+ ...
```

If the node has not currently got a token buffered, we can go to the `ReceiveToken` procedure, see Section 5.1.8. After executing this procedure, we return to `Node`.

```
% Receive an incoming token, if we are not currently storing one.
((!hasToken) ->
    ReceiveToken(id, black, msgCount, seq, active, basicDone)
)
+ ...
```

If the node has not yet informed the `doneChecker` process that it is done, then `id` can choose to do so, after which we return to the main `Node` procedure.

```
% inform the doneChecker that this node will no longer send out basic messages.
((!basicDone) ->
        informDone(id)
        . Node(id, black, msgCount, seq, active, hasToken, tok, true)
)
```

Finally, we can add actions that report something about the current state in a given node, without changing the state. We use the actions `reportActive(id)` to report that `id` is currently active, and the action `reportSeq` to report seq_{id}, after which we return to `Node` *without changing any parameters of* `Node`.

```
(
    reportSeq(seq)
    . Node(id, black, msgCount, seq, active, hasToken,tok, basicDone)
)
+
(
    (active) ->
    (reportActive(id)
    . Node(id, black, msgCount, seq, active, hasToken,tok, basicDone))
)
+ ...
```

As discussed in Section 2.2.3.2, we cannot check formulas that use the *mu X. ...* operator if the model specification contains such `reportX` actions, as this results in states having self-loops.

5.1.6 msgBuff and tokenBuff

We create a `msgBuff` process for each combination of nodes `sndr` and `recv` (`sndr` $\neq$ `recv`). If the `sndr` node does the matching `sendToBuff` action, then we add the received basic message `seq` to the sorted list of messages `b`. Similarly, for some `seq` that is already in our list `b`, `recv` can do the matching `recFromBuff` action to receive this message, and we remove this message from `b`.

Finally, if `b` is empty, this `msgBuff` can inform the `doneChecker` that its buffer `b` is empty. Unlike in `Node`, we do not set a boolean to indicate that we are *done* or that we have informed `doneChecker`. The `informEmpty` action can only be taken after *all* nodes have become *done*. As such, we know that `sndr` will not send any basic messages into this `msgBuff`, so it will also remain empty once it becomes empty. (See Section 5.1.7.)

```
msgBuff(b:MsgBuff, sndr:ID,recv:ID) = sum seq:Nat .
    ( % Receive a message from sndr.
        (recToBuff(seq, sndr, recv) . msgBuff( add(b, seq), sndr, recv) )
    + % Send a message on to recv.
        ( (seq in b) ->  sendFromBuff(seq, sndr, recv)
                        . msgBuff( remove(b, seq), sndr, recv) )
    )
    + % if the buffer is empty (and if the doneC can receive this)
      % inform it that buffer is empty.
    ( (b == []) -> informEmpty(sndr, recv) . msgBuff(b, sndr, recv))
;
```

The `tokenBuff` process is effectively the same as `msgBuff`, but our actions on `seq:Nat` are now on `t:Token`.

```
tokenBuff(tb:TokBuff, sndr:ID,recv:ID) = sum t:Token . (
    % Receive a token
    (tokRecToBuff(t, sndr, recv) . tokenBuff( addT(tb, t), sndr, recv ) )
    + % Send a token on.
    ( (t in tb) ->  tokSendFromBuff(t, sndr, recv)
                    . tokenBuff( removeT(tb,t), sndr, recv) )
);
```

5.1.7 doneChecker

Similar to the definitions related to `ID`, we will need to expand `doneChecker`'s specification for larger instances of N, by adding actions for the corresponding added nodes and message

buffers.

First, the `doneChecker` is informed by each of the nodes that this node has become *done*. After this, the `doneChecker` is informed by each message buffer when this buffer becomes empty.

When the full network has become *done*, the `doneChecker` moves on to `doneChecker_B`, which can repeatedly perform the `informExtraTokSafe` action, which informs node 0 that it can pass on the token after reaching the limit of iterations S.

In order to reduce the state space, we set a fixed order in which nodes and message channels must inform the `doneChecker` that they are done/empty. By setting a fixed order, we avoid a larger state space that would be caused by allowing nodes to become *done* in any chosen order. For any trace in our model where we take some actions to become *done* while working on the basic and control algorithm, we can instead choose to delay these actions until node 0 requires permission from the `doneChecker`, and this would not change the trace of basic and control actions.[1]

```
doneChecker =
    recvDoneID(Zero) . recvDoneID(One) .
    recvEmpty(Zero,One) . recvEmpty(One,Zero) .
    basicAlgoDone . doneChecker_B;

doneChecker_B = informExtraTokSafe . doneChecker_B;
```

5.1.8 ReceiveToken

We divide ReceiveToken$_i$ 4 up into two separate sections. First, the `ReceiveToken` procedure, is for taking in an incoming token. At the $wait(passive_i)$ call (line 1), we split off the procedure. Either we return to `Node` if the node is currently active, or we proceed to `HandleToken`, which carries out the rest of ReceiveToken$_i$.

If the token has done S iterations, node 0 must get permission from the `doneChecker` before we can receive the token. By making `recvTokSafe` the *first* action we do, we ensure that we only enter the `ReceiveToken` procedure if the network is actually *done*,[2] following the approach set out in Section 4.1.3.

[1] As an example, take some trace in the model where nodes 0 and 1 immediately become *done*, then we have some trace of basic and control actions a until we reach the point where we need permission from the *doneChecker* to continue. Clearly, making nodes 0 and 1 *done after* this trace a would have no impact on a, as clearly a did not involve these two nodes sending any basic messages.

[2] Else, if we for example could first receive the token and *then* tried to get permission from *doneChecker*, we would encounter a deadlock if node 0 itself had not yet become *done*.

In `ReceiveToken`, we use the `sum` operator to allow us to receive any configuration of the token. Then, `ReceiveToken_1` either sends us back to `Node` with `hasToken` set to True, or we proceed to `HandleToken`.

```
ReceiveToken(id:ID, black:ID, msgCount:Int, seq : Nat, active:Bool,
    basicDone:Bool) =
    (id == Zero && (seq >= SEQ_LIMIT) )
    % if we are at the limit of iterations we can freely do, first
    % get permission before we can go receive the actual token.
    -> (recvTokSafe . sum newT:Token .
        (tokRecFromBuff(newT, tokPrev(id), id)
        . ReceiveToken_1(id, black, msgCount, seq, active, basicDone, newT) ))
    <> (     sum newT:Token .
        (tokRecFromBuff(newT, tokPrev(id), id)
        . ReceiveToken_1(id, black, msgCount, seq, active, basicDone, newT) ))
;
ReceiveToken_1(id:ID, black:ID, msgCount:Int, seq : Nat, active:Bool,
    basicDone:Bool, newT:Token) =
    (active)
    % If the node is currently active, we return to Node. When we become
    % passive, we will then handle the token.
    -> (Node(id, black, msgCount, seq, active, true, newT, basicDone))
    <> (HandleToken(id, black, msgCount, seq, newT, basicDone))
;
```

5.1.9 HandleToken

Once the node is passive and has the token, we proceed to `HandleToken`. In the code for HandleToken$_i$ 4, we update various state variables. mCRL2 does not allow us to update variables within the same procedure. Instead, we update these values at the end of the procedure call.

We split on the same condition as in line 3 of HandleToken$_i$ 4: if $count_t$, defined here as `localMsgCount+countT(tok)`, equals 0, and $black_i$, defined here as `furthest(id, localBlack, blackT(tok))`, equals `id`, then we have detected termination, and node `id` can perform the `Announce(id)` action. After the `Announce` action, we return to `Node`.

Else, we have not yet found termination, and we send the token to the next node. We construct a new token, by setting $count_t$ and by using `newTokFurthest` to set $black_t$ using

$black_i = furthest_i(black_i, black_t)$. (see Section 5.1.3.) We then also update $black_{id}$, seq_{id}, and $count_{id}$, and then return to `Node`.

```
%PRE: When we get here, the node is passive (not active) & has the token.
HandleToken(id:ID, localBlack:ID, localMsgCount:Int, seq:Nat, tok:Token,
    basicDone:Bool) =
    ( localMsgCount + countT(tok) == 0
        && furthest(id, localBlack, blackT(tok)) == id )
    -> % We have terminated.
        (Announce(id) . Node(id, id, 0, seq, false, false, EMPTYTOK, basicDone))
    <> % Not yet terminated, continue on with the token.
        ( tokSendToBuff(pair(localMsgCount + countT(tok),
            newTokFurthest(id, localBlack, blackT(tok))), id, tokNext(id) )
            . Node(id,id, 0, seq+1, false, false, EMPTYTOK, basicDone)
        )
;
```

5.1.10 Initial configuration

Finally, we come to the initial configuration. Using the `comm` block, we set a series of communicating actions, such as `sendToBuff|recToBuff -> toBuff`. We then specify with the `allow` block that the communicating actions (such as `toBuff`) specifically *are* allowed, while specifying with the `hide` block that the individual actions that form a communicating action (such as `sendToBuff` and `recToBuff`) are specifically *not* allowed. This way, our model can do the communicating actions, so for example we allow a node to send a basic message to a buffer and simultaneously that this buffer receives the message, but can*not* do the individual actions separately.

```
init
    hide ({sendToBuff, recToBuff, sendFromBuff, recFromBuff,
           tokSendToBuff, tokRecToBuff, tokSendFromBuff,
           tokRecFromBuff, informDone, recvDoneID, informEmpty,
           recvEmpty, informExtraTokSafe, recvTokSafe},
        allow( {toBuff, fromBuff, tokToBuff, tokFromBuff,  Announce,
                BecomeActive, BecomePassive, nodeAlreadyActive,
                doneComm, emptyComm, tokSafeComm,
                cPermSend,
                ... %any reportX actions we may wish to include
                },
```

```
comm({sendToBuff|recToBuff -> toBuff,
      sendFromBuff|recFromBuff -> fromBuff,
      tokSendToBuff|tokRecToBuff -> tokToBuff,
      tokSendFromBuff|tokRecFromBuff -> tokFromBuff,
      informDone|recvDoneID -> doneComm,
      informEmpty|recvEmpty -> emptyComm,
      informExtraTokSafe|recvTokSafe -> tokSafeComm,
      sndPermSend|rcvPermSend -> cPermSend},
      ...
```

Once we have specified the permitted actions, we then specify our actual instance. As mentioned above, this varies depending on the actual instance: if we have more nodes, then we add more `Node`, `msgBuff` and `tokenBuff` processes as required.

Initially, we specify that node 0 has the token with $black_t = N - 1$ (specified with the constant `BLACK_INIT_ZERO`), and set `hasToken` to be false for all other nodes. We handle the initial token send slightly differently compared to Initialisation$_0$ 1: when node 0 becomes passive, it goes to the `handleToken` process. As $black_t$ is specified initially as $N - 1$, the $black_0 = 0$ condition will not hold, and as such we send on the token with $count_t \leftarrow count_0$.

Similarly, every node is initially active. Our model effectively includes specifications where some nodes are initially passive: as the first actions of such a trace, let these nodes perform `becomePassive`.

For the `msgBuff` processes: we create a process for each pair of nodes $i \neq j$. For the `tokenBuff` processes, we only create the processes to send the nodes from i to $(i + 1)\%N$.

```
   Node(Zero,Zero, 0,0, true, true, pair(0, BLACK_INIT_ZERO), false )
|| Node(One, One , 0,0, true, false, EMPTYTOK, false )
|| msgBuff([], Zero,One) || msgBuff([], One,Zero)
|| tokenBuff([], Zero,One) || tokenBuff([], One,Zero)
|| doneChecker
|| msgCounter(0)
```

5.2 Fault-tolerant model specification

We now discuss the changes made to the model for the fault-*tolerant* version of Safra's algorithm, presented in (37), again with $N = 2$. We will omit some of the parts of the model that are the same as in the previous specification. As before, see (58) for the full specification, as well as specifications for larger N.

Here, we allow for nodes to crash. Let C be the maximum number of nodes that may crash, specified in mCRL2 as the constant `MAX_N_CRASHES`. Given that we assume that at least 1 node will always remain non-crashed, we have that $0 \leq C < N$.

As discussed in Section 4.1.4, we provide the option to crash in the main `Node` procedure and as part of `ReceiveToken`.

5.2.1 Custom data types

We use the same custom data types as detailed in the previous section. We add two new data types, `Counter` and `CtrList`, and we alter the token to add the additional fields for this version of the algorithm.

We will leave out some redundant function definitions, such as getter functions, here.

Counter and CtrList

In the fault-sensitive model, we model $count_i$ for nodes i as a single integer `msgCount`. (See Section 5.1.5).

In the fault-tolerant version of the algorithm, $count_i$ is no longer a single integer, but instead a list of counters, one for each node in the network. As such, we create the `Counter` and `CtrList` data types:

```
% Each node keeps a list of (ID, counter_ID) for each ID (including its own).
% Similarly, the token also has such a list.
sort
 Counter = struct pair(fst:ID, snd:Int); % (ID, counter_ID)
 CtrList = List(Counter);
```

Each node and the token has one `CtrList`, with an element for each ID. Each `Counter` consists of an identifying ID, and a counter value for that ID $count_{id}$.

We then specify a number of functions for `CtrList`, as well as a constant:

```
map
 INIT_LIST : CtrList;
 % Constructor
 Ctr : ID # Int -> Counter;
 % Getter functions
 getIdC  : Counter -> ID;
 getCtrV : Counter -> Int;
 incrAtID : ID # CtrList -> CtrList;
 decrAtId : ID # CtrList -> CtrList;
```

```
 % Given a set of crashed nodes: sum over the list,
 % only adding the values from non-crashed nodes.
 sumNonCrashed : FSet(ID) # CtrList -> Int;
 % Given an ID, find the corrosponding element in the list
 % and replace snd with the provided Int.
 updateCount : ID # CtrList # Int -> CtrList;
var
 idAlter : ID;
 ctr : Counter;
 cl : CtrList;
 fstCtr : ID;
 sndCtr : Int;
 crashedN : FSet(ID);
 newCount : Int;
```

The constant `INIT_LIST` is dependent on the number of nodes: for each node (and so for each value in the `ID` data type), we add an element `(id, 0)`.

```
eqn
 % If we increase N, we need to add the corrosponding (id, 0) to this list.
 INIT_LIST = [pair(Zero,0), pair(One,0)];
```

As before, we introduce a constructor for `Counter`, and specify getter functions.

```
 % Constructor
 Ctr(fstCtr, sndCtr) = pair(fstCtr, sndCtr);
 ...
```

Given a specific id `idAlter`, we can increment (when we send a message to `idAlter`) or decrement (when we receive a message from `idAlter`) that `Counter` in the list with `incrAtId` and `decrAtId` respectively.

```
 incrAtID(idAlter, []) = [];
 (idAlter == getIdC(ctr))
        -> incrAtID(idAlter, ctr |> cl)
            = ( Ctr(getIdC(ctr), getCtrV(ctr)+1 ) ) |> cl;
 (idAlter != getIdC(ctr))
        -> incrAtID(idAlter, ctr |> cl)
            = ctr |> incrAtID(idAlter, cl);
 decrAtId(idAlter, []) = [];
 (idAlter == getIdC(ctr))
```

```
       -> decrAtId(idAlter, ctr |> cl)
           = ( Ctr(getIdC(ctr), getCtrV(ctr)-1 ) ) |> cl;
(idAlter != getIdC(ctr))
       -> decrAtId(idAlter, ctr |> cl)
           = ctr |> decrAtId(idAlter, cl);
...
```

For the two for-loops in ReceiveToken$_i$ 10, we need to be able to sum up all values $count_*^j$ for nodes $j \notin Crashed_i$. We use `sumNonCrashed` for this purpose.

```
% for sumNonCrashed, we only want to add the values of nodes
   % that are NOT in the crashedNode set.
sumNonCrashed(crashedN, []) = 0;
(getIdC(ctr) in crashedN)
       -> sumNonCrashed(crashedN, ctr|>cl)
           =                sumNonCrashed(crashedN, cl);
!(getIdC(ctr) in crashedN)
       -> sumNonCrashed(crashedN, ctr|>cl)
           = getCtrV(ctr) + sumNonCrashed(crashedN, cl);
...
```

The function `updateCount` looks for the element in the list with the id `idAlter`, and sets its count to `newCount`.

```
updateCount(idAlter, [], newCount) = [];
(idAlter == getIdC(ctr))
       -> updateCount(idAlter, ctr |> cl, newCount)
           = ( Ctr(idAlter, newCount) ) |> cl;
(idAlter != getIdC(ctr))
       -> updateCount(idAlter, ctr |> cl, newCount)
           = ctr |> updateCount(idAlter, cl, newCount);
```

Token for the tolerant model

In the fault-tolerant version of the algorithm, our token now carries considerably more values: first, $count_t$ is now a list of counters; furthermore, we add the set of nodes $Crashed_t$ to the token, as well as the sequence counter seq_t. This is reflected in the updated definition for `Token`:

```
sort                    % (count_t, black_t, crashed_t, seq_t)
 Token    = struct quad(fst: CtrList, snd:ID, trd:FSet(ID), fth:Nat);
 TokBuff  = List(Token);
```

Given this altered definition, we add getter functions, and adjust the constructors.

```
map
 Tok : CtrList # ID # FSet(ID) # Nat -> Token;
 countT : Token -> CtrList;
 blackT : Token -> ID;
 seqT   : Token -> Nat;
 crashedT  : Token -> FSet(ID);
 % updates crashed_t by REMOVING the second set from the token.
 updateCrTMin  : Token # FSet(ID) -> Token;
 % updates crashed_t by ADDING the second set to the token.
 updateCrTPlus : Token # FSet(ID) -> Token;
 % updates count_t^i, for the given ID, with value [INT].
 updateCoT : Token # ID # Int -> Token;
 incrSeqT  : Token -> Token;
 setSeqT : Token # Nat -> Token;
 updateBlackT : Token # ID -> Token;

 removeT: TokBuff # Token -> TokBuff;
 addT   : TokBuff # Token -> TokBuff;

 tokPrev, tokNext : ID -> ID;
 EMPTYTOK : Token;
 INIT_OLD_TOK : ID -> Token;
var
 fstV : CtrList;
 secV : ID;
 trdV : FSet(ID);
 fthV : Nat;
 crashI : FSet(ID);
 newSeq : Nat;
 tokX, tokT : Token;
 tokB : TokBuff;
 idCurr : ID;
 newCount : Int;
```

Given a set of nodes `crashI`, we update $Crashed_t$ with the `updateCrTMin` and `updateCrTPlus` functions:

```
eqn
```

```
% Constructor for a token:
Tok(fstV, secV, trdV, fthV) = quad(fstV, secV, trdV, fthV);
...
% crashed_t <- crashed_t \setdiff crashedSet
updateCrTMin(quad(fstV, secV, trdV, fthV), crashI)
       = quad(fstV, secV, (trdV - crashI ), fthV);
% crashed_t <- crashed_t \union crashedSet
updateCrTPlus(quad(fstV, secV, trdV, fthV), crashI)
       = quad(fstV, secV, (trdV + crashI ), fthV);
...
```

Given an id `idCurr` and a number `newCount`, we update $count_t$ by setting $count_t^i$ to `newCount`, using the `updateCount` function discussed above:

```
% count_t is unaltered, except count_t[idCurr] now = newCount.
updateCoT(quad(fstV, secV, trdV, fthV), idCurr, newCount)
       = quad(updateCount(idCurr, fstV, newCount), secV, trdV, fthV);
...
```

When the token is about to start a new iteration, either by returning to 0 or by travelling to the next alive node if 0 has crashed, we can increment seq_t with `incrSeqT`. Alternatively, in FailureDetector$_i$ 8, we may set seq_t to a specific value seq_i, for which we use `setSeqT`. Similarly, we use `updateBlackT` to set $black_t$.

```
% Increment seq_t
incrSeqT(quad(fstV, secV, trdV, fthV))
       = quad(fstV, secV, trdV, fthV+1);
% Set seq_t to a specific value.
setSeqT(quad(fstV, secV, trdV, fthV), newSeq)
       = quad(fstV, secV, trdV, newSeq);

updateBlackT(quad(fstV, secV, trdV, fthV), idCurr)
       = quad(fstV, idCurr, trdV, fthV);
...
```

Finally, we define constants: `EMPTYTOK` sets $count_t$ to the `INIT_LIST` constant described above, $black_t$ to node 0, $Crashed_t$ to be the empty set, and seq_t to 0.

In the fault-tolerant algorithm, nodes need to store the *previous* token that they handled. If for example node 0 crashes before sending *any* token out, node $N-1$ (or the first non-crashed node to which this applies) needs to send out the initial token as part of

FailureDetector$_i$ 8, so that the control algorithm gets started. This token is the same as `EMPTYTOK`, except we specify that $black_t$ is set to the node sending out this token (Line 8).

```
EMPTYTOK = quad(INIT_LIST, Zero, {}, 0);

% Initially, we set black_t = i for each node i.
% If we node 0 crashes before sending the first token, N-1 needs to take over.
% (etc if N-1 crashes.)
% In FD_i, we then send out a 'new' token, with black_t set to i.
INIT_OLD_TOK(secV) = quad(INIT_LIST, secV, {}, 0);
```

5.2.2 Node

Similar to the token, we have additional parameters for each `Node` process. First, we now store $count_{id}$ as a list of counters `mCtrList`. Next, we now store multiple tokens in the node: `oldTok` is the last token that this node has handled (and sent to $next_{id}$) or `INIT_OLD_TOK(id)`, possibly with some nodes added to $Crashed_t$. (See Section 5.2.3.)

As in our fault-sensitive specification (see Section 5.1.5), `hasToken` is true when this node has a token `newTok` buffered. We only buffer such a token if $seq_t = seq_{id} + 1$, as per the first condition in HandleToken$_{id}$ 10.

Next, we use finite sets `report` and `crashedSet` to store $report_{id}$ and $Crashed_{id}$ respectively. With `next`, we keep track of the next alive node in the ring to which we will send the token.

Finally, we have a boolean `isOnlyNodeAndAnnounced` that is only set to true if all other nodes have crashed, and we have done `Announce` through Line 6 of NewSuccessor$_{id}$ 9.

```
Node(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
        oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
        report:FSet(ID), crashedSet:FSet(ID), next:ID,
        isOnlyNodeAndAnnounced:Bool) = ...
```

`Node` has the same set of choices as in the sensitive model.

We add the additional requirement in ReceiveBasicMessage$_{id}$ 7: as per Line 1, we may not receive basic messages from a node j if $j \in Crashed_{id}$, so if we know that j has crashed:

```
( sum j:ID . (j != id && !(j in crashedSet) ) -> sum newSeq:Nat .
    (recFromBuff(newSeq,j, id) . ...
)
+
...
```

We apply a similar restriction on *sending* basic messages, in line with SendBasicMessage$_{id}$ 6, which we omit here.

We now add four additional options to `Node`:

First, we can run FailureDetector$_{id}$ 8 using the `localFD` procedure. Upon completing the procedure, this will return to the `Node` procedure. (See Section 5.2.3.)

```
(
 localFD(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
               basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
 % localFD will return to Node.
)
+
...
```

Secondly, part of our definition of *done* is the following: a message buffer may become *done* ... when the receiving node knows that the sender has crashed. Node `id` may *not* receive messages from a node $j \in$ `crashedSet`. As such, instead of the message buffer (j, id) informing the `doneChecker` that it is empty, we can instead have node `id` inform the `doneChecker` that this message buffer is *done*. (See Section 5.2.5.)

```
(% if id knows that a sender j has crashed (so j \in crashed_id),
 % then this node id will not read from this channel.
 % For the doneChecker, we can thus act as though this channel is empty.
 % Note specifically that we do NOT do this if j is only in report,
 % as we can still read from j as long as j is only in report.
 sum j:ID . (j in crashedSet && !isOnlyNodeAndAnnounced)
  -> (sRecKnowsCr(j,id)
  . Node(id, black, mCtrList, seq, active, oldTok,hasToken,newTok, basicDone,
                  report, crashedSet, next, isOnlyNodeAndAnnounced) )
)
+
...
```

Thirdly, similar to the `reportActive` and `reportSeq` actions, we add an action `reportErrorIDinCrashed`. If this action can be taken at a node `id`, then `id` $\in Crashed_{id} \cup Report_{id} \cup Crashed_t$.

```
(
    (id in (report+crashedSet+crashedT(newTok))) ->
    (reportErrorIDinCrashed(id)
```

```
    . Node(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
            basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced))
)
+
...
```

Finally, we allow the node to crash. Unlike all the other options, we do *not* return to `Node`: after performing this action, so that `id` cannot perform any further actions.

```
( crash(id) )
+
...
```

5.2.3 The perfect failure detector

As discussed in Section 4.1.1, we split the perfect failure detector, across one global process `globalFD`, and a process `localFD` which each `Node` can choose to use.

globalFD

The process `globalFD` stores a finite set `crashedNodes` (initially $\emptyset$) of nodes that have crashed.

When a node `i` crashes, its crashing action `crash(i)` communicates with `globalFD`'s `fdDetect(i)` action. This means that node `i` can only crash with `globalFD`'s knowledge. We specify that `fdDetect(i)` can only occur if 1) `i` is not already in `crashedNodes`, so `i` has not already crashed, and 2) if the size of `crashedNodes` is smaller than `MAX_N_CRASHES`. As a result, a node `i` can only crash *once*, and we ensure that no more than `MAX_N_CRASHES` crashes occur in the system.

Similar to the `sRecKnowsCr` action that `Node` can run, the global failure detector can also communicate with the `doneChecker`: a message buffer may become *done* .... when the receiving node has crashed. Further, a node `i` is deemed to be *done* when it crashes. As such, when the `doneChecker` needs to be informed about either a node `i` being done or a message buffer (j, i) being done, the `globalFD` process can inform `doneChecker` using the `sCrDone(i)` action. (See Section 5.2.5.)

Finally, the main purpose of failure detector in our system: using the `sInformCrashed` action, `globalFD` can inform a non-crashed node `j` that node `i` has crashed.

```
globalFD(crashedNodes:FSet(ID)) = sum i:ID . (i in crashedNodes)
->
```

```
  ( (sCrDone(i) . globalFD(crashedNodes))
  +
  (sum j:ID . (i != j && !(j in crashedNodes))
         -> (sInformCrashed(j,i) . globalFD(crashedNodes)) )
     )
 <> ((# crashedNodes < MAX_N_CRASHES)
            -> (fdDetect(i) . globalFD(crashedNodes+{i}) ))
 ;
```

localFD

In the `localFD` process, we execute FailureDetector$_{id}$ 8. Similar to the `ReceiveToken` specifications, we split the procedure up over a number of different processes.

The process starts by being informed, by `globalFD`, that some other node `j` has crashed, and we add `j` to `report`. Following FailureDetector$_{id}$, if the crashed node `j` was $next_{id}$, then we move to the `newSucc_FD` process (see Section 5.2.4), which in turn moves to the `localFD_6` process to continue executing FailureDetector$_{id}$. Else we simply return to `Node`.

```
localFD(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
        oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
        report:FSet(ID), crashedSet:FSet(ID), next:ID,
        isOnlyNodeAndAnnounced:Bool) =
    sum j:ID . (!(j in crashedSet+report)) -> (
        rInformCrashed(id, j)
        . (j == next)
        -> (newSucc_FD(id, black, mCtrList, seq, active,
                       oldTok,hasToken,newTok, basicDone,
                       (report+{j}), crashedSet, next,
                       isOnlyNodeAndAnnounced))
        <> (Node(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
                basicDone, (report+{j}), crashedSet, next,
                isOnlyNodeAndAnnounced)) % simply add j to report_i & continue.
        )
;
```

After working through NewSuccessor$_{id}$ 9, we return to Line 6 of FailureDetector$_{id}$ with process `localFD_6`. If `seq > 0 || next < id` is *false*, we simply return to `Node` at this point, with an updated $next_{id}$.

Else, we update $Crashed_t$ with `updateCrTPlus` and $black_t$ with `updateBlackT`. If the token is starting a new iteration of the ring (`next < id`), then we also set seq_t to $seq_{id}+1$. Then, we move to `localFD_11`.

```
localFD_6(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
          oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
          report:FSet(ID), crashedSet:FSet(ID), next:ID,
          isOnlyNodeAndAnnounced:Bool) =
    (seq > 0 || next < id)
    -> ( (next < id)
        -> (localFD_11(id, black, mCtrList, seq, active,
                setSeqT(updateBlackT(updateCrTPlus(oldTok, report), id), seq+1),
                hasToken,newTok, basicDone, report, crashedSet, next,
                isOnlyNodeAndAnnounced))
        <> (localFD_11(id, black, mCtrList, seq, active,
                        updateBlackT(updateCrTPlus(oldTok, report), id),
                hasToken,newTok, basicDone, report, crashedSet, next,
                isOnlyNodeAndAnnounced) )
    )
    <> (Node(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
             basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced) )
;
```

In `localFD_11`, we simply carry out Line 11 of FailureDetector$_{id}$: we take `oldTok`, which was the last token handled by `id` and now has updated values $black_t$, $Crashed_t$ and potentially seq_t, and send the token on to $next_{id}$. After this, we return to the main `Node` process.

```
localFD_11(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
        oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
        report:FSet(ID), crashedSet:FSet(ID), next:ID,
        isOnlyNodeAndAnnounced:Bool) =
    tokSendToBuff(oldTok, id, next)
    . Node(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
             basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
    ;
```

5.2.4 NewSucc

Both FailureDetector$_{id}$ and ReceiveToken$_{id}$ call NewSuccessor$_{id}$, and so we do the same in `localFD` and `HandleToken_*`. In `localFD`, we are guaranteed to do an action *before* accessing the NewSuccessor$_{id}$ procedure (namely `informCrashed`), but we return to `Node`, after leaving NewSuccessor$_{id}$, without taking further actions. In `HandleToken_*`, we are guaranteed to do some action *after* accessing the NewSuccessor$_{id}$ procedure (namely either `Announce` or sending the token on to the next node), but we might not do any actions prior to entering NewSuccessor$_{id}$.

mCRL2 does not allow for (theoretically possible) unguarded recursion, where some process can access itself without taking an action.[1] As a result, we cannot use the same `NewSuccessor` procedure for both `localFD` and `HandleToken_*`, else there would be a *theoretical* path from `Node` to `NewSuccessor` via `HandleToken_*`, and then from `NewSuccessor` back to `Node` via `localFD`, all without actually taking an action.

We will discuss `newSucc_FD` and leave out `newSucc_HT`. These are functionally equivalent; the only difference between the two is the parameters which we pass to them.[2]

When we call `newSucc_FD`, we pass the variable `oldNext` to it, where `oldNext` is the node that we have just detected to have crashed. We then immediately move on to `newSucc_4_FD`, adjusting $next_{id}$: the `getNextID` function carries out the while-loop as specified in the opening lines of NewSuccessor$_{id}$ 9. This function keeps iterating until it finds a node that, to our current knowledge, has not crashed.[3]

```
% in the eqn section:
%PRE: there is at least one viable ID *not* in crashedNodes.
(id in crashedNodes)  -> getNextID(id, crashedNodes)
    = getNextID( NatToID((IDtoNat(id)+1)mod N_PROCS) , crashedNodes);
!(id in crashedNodes) -> getNextID(id, crashedNodes)
    = id;
...
% in the proc section:
```

[1] Take the following process specification as an example: $A = (false) \rightarrow A <> (a \ . \ A)$;, where a is an action. Clearly, the unguarded recursion will never take place, as this would require *false* to be true. Nonetheless, mCRL2's lineariser does not allow this specification.

[2] As an example, *handleToken* does not keep track of the previous token *oldTok*, so we also do not pass this to *newSucc_HT*.

[3] As noted in a comment, we assume that at least one node is not in our set of crashed nodes. If this was violated, and $Crashed_i \cup Report_i$ = the set of all nodes, then NewSuccessor$_{id}$ 9 would stall as it infinitely iterates over the while-loop. Similarly, it would not be possible to generate the state space for our model, as mCRL2 would stall trying to find the successor node with *getNextID*.

```
% oldNext = the j that crashed, and that (pre-call) equals next_i.
newSucc_FD(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
        oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
        report:FSet(ID), crashedSet:FSet(ID),
        oldNext:ID, isOnlyNodeAndAnnounced:Bool) =
    newSucc_4_FD(id, black, mCtrList, seq, active, oldTok,hasToken,newTok,
            basicDone, report, crashedSet,
            getNextID(
                NatToID((IDtoNat(oldNext)+1) mod N_PROCS) ,
                (crashedSet+report)),
            isOnlyNodeAndAnnounced);
```

Once we have a new value for $next_{id}$, we can continue on to `newSucc_4_FD`.

If we find that $next_{id}$ equals `id`, then we can find termination merely from the local node. If this node was not yet passive, we perform `BecomePassive`, and then this node can `Announce` [1], after which we return to `Node`, with `isOnlyNodeAndAnnounced` set to true to ensure that the node does not try to send a token to itself. (See Section 5.2.7.)

Else, we potentially update $black_{id}$, and then return to `localFD_6`.[2]

```
% If we have only 1 node remaining, we can Announce and return to Node.
newSucc_4_FD(id:ID, black:ID, mCtrList:CtrList, seq : Nat, active:Bool,
                oldTok:Token,hasToken:Bool,newTok:Token, basicDone:Bool,
                report:FSet(ID), crashedSet:FSet(ID), next:ID,
                isOnlyNodeAndAnnounced:Bool) =
    (next == id)
    -> ( lastNodeRemaining(id) . (active)
            -> (BecomePassive(id) . Announce(id)
                . Node(id, id, mCtrList, seq, false,EMPTYTOK,false,EMPTYTOK,
                    basicDone, {}, (report+crashedSet), next, true))
            <> (                    Announce(id)
                . Node(id, id, mCtrList, seq, false,EMPTYTOK,false,EMPTYTOK,
                    basicDone, {}, (report+crashedSet), next, true))
       )
    <> ( (black == id)
            -> localFD_6(id, black, mCtrList, seq, active,
                    oldTok,hasToken,newTok, basicDone, report, crashedSet,
```

[1] In $newSucc\_4\_HT$, we do not need to check if the node is active. A node running $HandleToken$ here already implies that the node was passive.

[2] In $newSucc\_4\_HT$, we then return to $HandleToken\_18$.

```
                    next, isOnlyNodeAndAnnounced)
            <> localFD_6(id, furthest(id, black, next), mCtrList, seq, active,
                    oldTok,hasToken,newTok, basicDone, report, crashedSet,
                    next, isOnlyNodeAndAnnounced)
        )
;
```

Note that, if node `id` is the last node remaining, we set $Report_{id}$ to the empty set `{}`, and set $Crashed_{id}$ to $Crashed_{id} \cup Report_{id}$. This adjustment is not part of the *original* NewSuccessor$_{id}$ procedure 9 as set out in (37). We will discuss in Section 7.1 why this change is necessary.

5.2.5 doneChecker

We split the `doneChecker` procedure up into multiple components, starting with `doneChecker_A`. As with the `doneChecker` in the fault-sensitive specification, we need to expand the specification if we add nodes and matching message channels to the network.

Similar to Section 5.1.7, we first check that *nodes* are *done*. Either a node i does the `becomeDone` action, in which case `doneChecker_A` does the corresponding `recvDoneID` action, or i has crashed, and `doneChecker_A` is informed by `globalFD` with the `CrDone` actions.

```
doneChecker_A =
    (recvDoneID(Zero) + rCrDone(Zero))
    . (recvDoneID(One) + rCrDone(One))
    . doneChecker_B;
```

Once we have determined that all nodes are *done*, `doneChecker_B` sets out to determine that all message buffers are *done*. For each message buffer, the `doneChecker` can find out it is *done* in three ways: first, as before, if the message buffer is empty, it can inform `doneChecker_B`; secondly, if the receiver has crashed, `globalFD` can inform the `doneChecker` of this with the `CrDone` actions; finally, if the sender has crashed, the receiver, once it knows about the sender's crash, can inform the `doneChecker` with the `RecKnowsCr` actions.

```
doneChecker_B =
    % A channel must be empty, the receiver must have crashed,
    % or the receiver must know that the sender has crashed, before we
    % can determine that the channel is done.
```

```
    (recvEmpty(Zero,One) + rCrDone(One) + rRecKnowsCr(Zero,One))
    . (recvEmpty(One,Zero) + rCrDone(Zero) + rRecKnowsCr(One, Zero))
    . basicAlgoDone . donechecker_C;
```

Once `doneChecker_B` has determined that all message buffers are also empty, we move on to `doneChecker_C`. Here, we can repeatedly send out permission to nodes to continue with the control algorithm after hitting the limit of iterations S.

```
donechecker_C = informExtraTokSafe . donechecker_C;
```

5.2.6 ReceiveToken

Similar to the fault-sensitive specification in Section 5.1.8, we split ReceiveToken$_i$ up into multiple processes. To start, we call `ReceiveToken`.

As in the fault-sensitive specification, we first check `seq >= SEQ_LIMIT`, and request permission from the `doneChecker` if this condition is true. However, there are two significant differences here:

First, we now force *every* node to get permission from the `doneChecker`, not just node 0. If node 0 crashes, we still want to enforce our limit on the number of iterations.

Secondly, after receiving permission from the `doneChecker`, we can instead choose to run the `localFD` procedure. If node `id` receives permission to receive the token, it can receive the token from *any* node `j`, including nodes that have crashed since sending the token. If there is some other alive node `j` in the network, and `id` does not have the token, then we expect that `j` will eventually send the token to `id` if `j` does not detect termination. However, if there is no incoming token as a result of all nodes $\neq$ `id` *having crashed*, we must be able to access `localFD` to detect this.[1]

```
% PRE: The process did not previously have a token,
% so hasToken=False in the Node that called this.
ReceiveToken(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat, active:Bool,
            oldTok:Token, basicDone:Bool, report:FSet(ID), crashedSet:FSet(ID),
            next:ID, isOnlyNodeAndAnnounced:Bool) =
(seq >= SEQ_LIMIT)
    -> (recvTokSafe
        . (localFD(id, localBlack, localCtrList, seq, active,
```

[1] If we are not yet at our limit S, we do *not* need to add the option to go to *localFD* here. As discussed in Section 4.1.3, the first action on the path when $seq_{id} < S$, is the actual *tokRecFromBuff* action. If there is no incoming token, then we also cannot do this action, and so we are free to access *localFD* through the normal main node procedure.

```
                    oldTok,false,EMPTYTOK, basicDone, report, crashedSet,
                    next, isOnlyNodeAndAnnounced)
                    % localFD will return to Node. NOTE: hasToken=false,
                    % as we have not actually received a token.
            + sum candidateNewTok:Token . sum j:ID
                . (tokRecFromBuff(candidateNewTok, j, id) .
                   ReceiveToken_1(id, localBlack, localCtrList, seq, active,
                    oldTok, candidateNewTok, basicDone, report, crashedSet,
                    next, isOnlyNodeAndAnnounced) )
          )
        )
    <> ( sum candidateNewTok:Token . sum j:ID
                . (tokRecFromBuff(candidateNewTok, j, id) .
                   ReceiveToken_1(id, localBlack, localCtrList, seq, active,
                        oldTok, candidateNewTok, basicDone, report, crashedSet,
                        next, isOnlyNodeAndAnnounced) )
        )
;
```

After receiving a token, we need to check seq_t. If $seq_t = seq_{id} + 1$, then we can receive the token and move on to `ReceiveToken_2A`.

Else, we dismiss the token. We expect that, for all tokens t and nodes i, $seq_t \leq seq_i + 1$. To check this property, we add the `reportErrorDismissTooNewToken` action, which can only be taken when a node encounters a token where $seq_t > seq_i + 1$.

```
ReceiveToken_1(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat, active:Bool,
            oldTok:Token, candidateNewTok:Token, basicDone:Bool, report:FSet(ID),
            crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    (seqT(candidateNewTok) == seq+1)
    -> (ReceiveToken_2A(id, localBlack, localCtrList, seq, active, oldTok,
            candidateNewTok, basicDone, report, crashedSet,
            next, isOnlyNodeAndAnnounced))
    <> ( ((seqT(candidateNewTok) < seq+1)
        -> dismissOldToken(candidateNewTok)
        <> reportErrorDismissTooNewToken(id)
        )
        . Node(id, localBlack, localCtrList, seq, active, oldTok,false,EMPTYTOK,
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
       )
```

```
;
```

Finally, `ReceiveToken_2A` splits on the $wait(passive_{id})$ call in ReceiveToken$_i$ 10. We either move on to `HandleToken` if the node is passive, or we return to the `Node` process.

As discussed in Section 4.1.4, we add an option for a node to crash instead of moving to `HandleToken`, if the node is currently passive.

```
ReceiveToken_2A(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat, active:Bool,
            oldTok:Token, newTok:Token, basicDone:Bool, report:FSet(ID),
            crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    (active)
    -> (Node(id, localBlack, localCtrList, seq, active, oldTok,true,newTok,
            basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced))
    <> (crash(id) +
        HandleToken(id, localBlack, localCtrList, seq, newTok, basicDone,
            report, crashedSet, next, isOnlyNodeAndAnnounced))
;
```

5.2.7 HandleToken

The HandleToken$_i$ procedure for the fault-tolerant algorithm 10 is considerably more involved than the fault-sensitive equivalent 4, and as such so is our specification. We create a series of procedures, where the N in `HandleToken_N` refers to the corresponding line in HandleToken$_i$ 10.

We begin with a series of updates to variables, with one update in each process.

```
%PRE: When we get here, the node is passive (not active)
%       & has the token & the token is not old (seq_t = seq_i + 1).
HandleToken(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % Note that we have previously, when receiving the newTok,
    % already verified that seq_t == seq_i+1.
    % update black_i
    HandleToken_3(id, furthest(id, localBlack, blackT(newTok)) , localCtrList,
            seq, newTok, basicDone, report, crashedSet,
            next, isOnlyNodeAndAnnounced);

HandleToken_3(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
```

```
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % update crashed_t.
    HandleToken_4(id, localBlack, localCtrList, seq,
            updateCrTMin(newTok, crashedSet), basicDone, report, crashedSet,
            next, isOnlyNodeAndAnnounced);

HandleToken_4(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % update crashed_i
    HandleToken_5(id, localBlack, localCtrList, seq, newTok, basicDone,
        report, (crashedSet+crashedT(newTok)), next, isOnlyNodeAndAnnounced);

HandleToken_5(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % update report_i
    HandleToken_6(id, localBlack, localCtrList, seq, newTok, basicDone,
        (report-crashedT(newTok)), crashedSet, next, isOnlyNodeAndAnnounced);
```

In `HandleToken_6`, we update $count_t^{id}$ if the condition is met as specified in Line 6. We use the `sumNonCrashed` function to add the $count_{id}^{j}$ values where $j \notin Crashed_{id}$, and then update $count_t$ with the `updateCoT` function.

We use the `addToCountT` and `skipAddToCountT` actions for model checking. As discussed in Section 4.3, if a node does the `skipAddToCountT` action, we generally expect the node to eventually do the `addToCountT` action.

```
HandleToken_6(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    (localBlack == id || report == {})
        -> (addToCountT(id) .
            HandleToken_10(id, localBlack, localCtrList, seq,
                updateCoT(newTok, id, sumNonCrashed(crashedSet, localCtrList)),
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
           )
        <> (skipAddToCountT(id) .
            HandleToken_10(id, localBlack, localCtrList, seq, newTok,
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
```

```
            %no change
        )
;
```

In `HandleToken_10` we again use `sumNonCrashed`, this time to sum up $count_t^j$. If the conditions on lines 10 ($black_{id} = id$) and 14 ($sum_i = 0$) both evaluate to true, then we do `Announce`, after which we return to the `Node` procedure.

```
HandleToken_10(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % We combine the if on line 10, and the summation on lines 11-13,
    % and the condition based on said summation on lines 14-15 here.
    (localBlack == id && (sumNonCrashed( crashedSet, countT(newTok) ) == 0 ) )
    -> ( (Announce(id)
         . Node(id, id, localCtrList, seq, false,EMPTYTOK,false,EMPTYTOK,
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced) ) )
    <> (HandleToken_16(id, localBlack, localCtrList, seq, newTok, basicDone,
                report, crashedSet, next, isOnlyNodeAndAnnounced) %no change
    );
```

If we do not `Announce`, we instead move on to `HandleToken_16`. If node `id` found out from the token that `next` has crashed, we move to `newSucc_HT`, which will then move to `HandleToken_18` (unless we `Announce` in `newSucc_HT`). Else, we move on directly to `HandleToken_18`.

```
HandleToken_16(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    (next in crashedT(newTok))
        -> (newSucc_HT(id, localBlack, localCtrList, seq, newTok, basicDone,
                report, crashedSet,
                next, isOnlyNodeAndAnnounced)) % newSuccessor will call HT_18.
        <>  (HandleToken_18(id, localBlack, localCtrList, seq, newTok,
                basicDone, report, crashedSet,
                next, isOnlyNodeAndAnnounced) ) %no change
        ;
```

Using `incrSeqT`, we increment seq_t if required.

```
HandleToken_18(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    (next < id)
        -> (HandleToken_20(id, localBlack, localCtrList, seq, incrSeqT(newTok),
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced))
        <> (HandleToken_20(id, localBlack, localCtrList, seq, newTok,
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced));
```

Depending on $Report_{id}$, we either update $Crashed_t$, $black_t$, $Crashed_{id}$ and $report_{id}$, or we only update $black_t$.

```
HandleToken_20(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    ( (report != {})
        -> (
            HandleToken_25(id, localBlack, localCtrList, seq,
                updateBlackT(updateCrTPlus(newTok, report), id),
                basicDone, {}, (crashedSet+report), next, isOnlyNodeAndAnnounced)
        )
        <> (HandleToken_25(id, localBlack, localCtrList, seq,
                updateBlackT(newTok, furthest(id, localBlack, next)),
                basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced))
    )
;
```

Finally, in `HandleToken_25`, we send on the token to $next_{id}$. However, if `isOnlyNodeAndAnnounced` is true, then this would involve `id` sending the token to itself, as $next_{id} = id$ in that scenario. To avoid this redundancy, we skip this send.

```
HandleToken_25(id:ID, localBlack:ID, localCtrList:CtrList, seq:Nat,
        newTok:Token, basicDone:Bool, report:FSet(ID),
        crashedSet:FSet(ID), next:ID, isOnlyNodeAndAnnounced:Bool) =
    % We do not send the token from id to itself.
    (!isOnlyNodeAndAnnounced) -> (
    tokSendToBuff(newTok, id, next)
    . Node(id, id, localCtrList, seq+1, false, newTok,false,EMPTYTOK,
            basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced)
    )
```

```
    <>
    ( Node(id, id, localCtrList, seq+1, false, newTok,false,EMPTYTOK,
            basicDone, report, crashedSet, next, isOnlyNodeAndAnnounced) )
    ;
```

5.2.8 Initial configuration

Similar to the initial configuration in 5.1.10, we specify a number of communicating actions for the fault-tolerant model, and block the individual actions.

```
comm({...
  % New for the Tolerant version:
  crash|fdDetect -> cCrash,
  sInformCrashed|rInformCrashed -> cInformCrashed,
  sCrDone|rCrDone -> cCrDone,
  sRecKnowsCr|rRecKnowsCr -> cRecKnowsCr}, ... }
```

As mentioned before, if we add additional nodes to the specification, we need to add the corresponding nodes and buffers to this initial configuration. In the fault-*sensitive* models, we only require token buffers from nodes i to $(i+1)\%N$. However, in the fault-tolerant model, we must have token buffers between *every* pair of nodes, as discussed in Section 2.1.3.1.

We instantiate the token in node 0 in accordance with Initialisation$_0$ 5, and we specify that `globalFD` initially has the empty set, to signify that there are no crashed nodes.

```
   Node(Zero,Zero, INIT_LIST, 0, true, INIT_OLD_TOK(Zero), true,
        quad(INIT_LIST,BLACK_INIT_ZERO, {}, 1), false, {},{}, One, false)
|| Node(One, One,  INIT_LIST, 0, true, INIT_OLD_TOK(One), false,
        EMPTYTOK, false, {},{}, Zero, false)
|| msgBuff([], Zero,One) || msgBuff([], One,Zero)
|| tokenBuff([], Zero,One) || tokenBuff([], One,Zero)
|| doneChecker_A
|| msgCounter(0)
|| globalFD({})
```

5.3 Properties in regular modal μ-calculus

Given that we have now created our mCRL2 specifications, we can now write our properties, described in words in Section 4.3, in mCRL2's regular modal μ-calculus.

Similar to our specifications, some of our formulas are limited to specific instances. We attach IDs to some actions, such as `Announce`. To specify 'taking any action *other than* `Announce`', we write $!Announce(Zero)$ && $!Announce(One)$ for $N = 2$. For larger N, we add the additional values && $!Announce(Two)$ && ... as necessary. We indicate with ... when our formulas are need to be expanded for larger N.[1]

In order to quantify over infinite data types (specifically, *forall* / *exists* $s : Nat$....), we need to limit s to a finite number of possible values. As discussed in Section 4.1.2, once we set a limit on the model with *done*, we know that there is an upper bound on the number of iterations that the control algorithm takes. Where possible, we take the higher upper bound of $s < S + N$ (Theorem 4), so that we can use the same formula for both the fault-sensitive and fault-tolerant instances. Thus, when we use quantifiers over s, we specify *forall* / *exists* $s : Nat.\ val\ (s < SEQ\_LIMIT + N\_PROCS) \implies$ ...

We start with the two properties put forward in (37):

- **Liveness**: If the basic algorithm has terminated, this termination is eventually detected by a (non-crashed) node.
  $[(!Announce(Zero)$ && $!Announce(One)$ && ...$) *\ .\ basicAlgoDone]$
  $mu\ X.\ [!Announce(Zero)$ && $!Announce(One)$ && ...$]X$ && $\langle true\rangle true$

  As mentioned in Section 4.1.2, we use `basicAlgoDone` as an approximation of basic algorithm terminating. It is possible for the model to detect termination before the `basicAlgoDone` action is taken, but *if* no node has done the `Announce` action when the network becomes *done*, we should always eventually have some node detect termination.

- **Safety**: When termination is detected, the basic algorithm has indeed terminated at some point in the past.
  forall $i : ID\ .\ [true *\ .\ (Announce(Zero) + Announce(One) + ...)\ .$
  $true *\ .\ (becomeActive(i) + reportActive(i))]false$

  Once termination is detected, the basic algorithm having terminated means all nodes are passive or crashed, and no node may become active, which would happen if a node i received a basic message.

Next, we specify our additional properties:

[1] The full formulas for larger N can be found in (58).

- **P0**: All traces in the model eventually reach `Announce`:
  $mu\ X.\ [!Announce(Zero)\ \&\&\ !Announce(One)\ \&\&\ ...]X\ \&\&\ \langle true\rangle true$

  Every state that has not yet Announced must have at least 1 possible action, and any action that we take where we do *not* do the `Announce` action must lead to a state in X.

- **P1**: All nodes i can perform the `Announce` action:
  $forall\ i : ID\ .\ \langle true * .Announce(i)\rangle true$

- **P2**: All nodes i can send a basic message to and receive a basic message from all nodes j, where $i \neq j$:
  $forall\ i : ID\ .\ \ forall\ j : ID\ .\ i \neq j\ \Longrightarrow$
  $(exists\ s1 : Nat\ .\ val\ (s1 < SEQ\_LIMIT + N\_PROCS)\ \Longrightarrow$
  $\langle true * \ .\ \ toBuff\,(s1, i, j)\rangle true)$
  $\&\&$
  $(exists\ s2 : Nat\ .\ val\ (s2 < SEQ\_LIMIT + N\_PROCS)\ \Longrightarrow$
  $\langle true * \ .\ \ fromBuff\,(s, i, j)\rangle true)$

- **P3**: In the fault-tolerant setting: All nodes i can crash:
  $forall\ i : ID\ .\ \langle true * \ .\ cCrash(i)\rangle true$

  Note that we use `cCrash` here, and not `crash`. `cCrash` is the *communicating* action in our model, and the `crash` action (in isolation) is not a possible in our models.

- **P4**: We can `Announce` *with* the network being *done*, specifically with the `doneChecker` granting permission to continue.
  $exists\ i : ID\ .\ \langle true * \ .\ tokSafeComm\ .\ true * \ .\ Announce(i)\rangle true$

  The `tokSafeComm` action can only be taken once the network is *done*, and once the token has completed `SEQ_LIMIT` iterations. Note that the `tokSafeComm` action is performed *after* `doneChecker` performs the `basicAlgoDone` action.

- **P5**: We can `Announce` *without* the network being done.
  $exists\ i : ID\ .\ \langle !(basicAlgoDone) * \ .\ Announce(i)\rangle true$

- **P6**: In the fault-tolerant setting: If a node i performs `Announce` and then crashes, we always have some other node j that does `Announce`.
  $forall\ i : ID\ .\ [true * \ .\ Announce(i)\ .\ true * \ .\ cCrash(i)]$
  $mu\ X\ .\ [!Announce(Zero)\ \&\&\ !Announce(One)\ \&\&\ ...]X\ \&\&\ \langle true\rangle true$

- **P7**: For each message m received by a node i: $seq_m \leq seq_i + 1$.
  $forall\ i : ID\ .\ [true*\ .\ reportErrorTooNewMessage(i)]false$

  In our model, we have the `reportErrorTooNewMessage` action that the model can take if this property is violated. If the action can never be taken, then the property holds.

- **P8**: In the fault-tolerant setting: For all tokens t received by a node i: $seq_t \leq seq_i + 1$.
  $forall\ i : ID\ .\ [true*\ .\ reportErrorDismissTooNewToken(i)]false$

- **P9**: In the fault-tolerant setting: i is never in $Crashed_i \cup Report_i \cup Crashed_t$.
  $forall\ i : ID\ .\ [true*\ .\ reportErrorIDinCrashed(i)]false$

- **P10**: In the fault-tolerant setting: If some node i does *not* update $count_t^i$ in the current iteration of the control algorithm, then at least one of the following will always eventually occur:

  - Node i updates $count_t^i$ in a later iteration.
  - Node i crashes.
  - Node i eventually is the final node remaining, and does *Announce* through line 6 of NewSuccessor$_i$ 9.

  $forall\ i : ID\ .\ [true*\ .\ skipAddToCountT(i)]$
  $mu\ X.\ [!addToCountT(i)\ \&\&\ !cCrash(i)\ \&\&\ !lastNodeRemaining(i)]X\ \&\&\ \langle true\rangle true$

  We use the `skipAddToCountT` action to indicate that i has decided *not* to update $count_t^i$ in some iteration. The three actions in the $[..]X$ then correspond to the three options listed above, where `lastNodeRemaining(i)` is performed in `newSucc_4_*` immediately before i performs `Announce(i)`.

- **P11**: In the fault-sensitive setting: seq_i may equal any value in the range $[0,\ S+1]$, and can*not* be higher than $S + 1$.
  $forall\ s : Nat\ .$
  $(val\ (s \leq SEQ\_LIMIT) \implies \langle true*\ .\ reportSeq(s)\rangle true)$
  $\&\&$
  $(val\ (s == SEQ\_LIMIT + 1) \implies [true*\ .\ reportSeq(s)]false)$

- **P12**: In the fault-tolerant setting: seq_i may equal any value in the range $[0,\ S + 1 + min(N-2, C)]$, and can*not* be higher than $S + 1 + min(N-2, C)$.

$forall\ s : Nat\ .$
$(val\ (s \leq SEQ\_LIMIT + min(N\_PROCS - 2, MAX\_N\_CRASHES))$
$\Longrightarrow\ \langle true*\ .\ reportSeq(s)\rangle true)$
&&
$(val\ (s == 1 + SEQ\_LIMIT + min(N\_PROCS - 2, MAX\_N\_CRASHES))$
$\Longrightarrow\ [true*\ .\ reportSeq(s)]false)$

As explained in Section 4.3, the upper bound is set by C or $N - 2$, which ever is smaller.

6

Evaluation

We have put our full artifact on Mendeley Data (58). Alternatively, we have put our specifications, formulas and *.py* scripts on GitHub:
`https://github.com/Andy-Tatman/FaultTolerantSafra-mCRL2-Artifact`.

6.1 Approach

System settings

In order to generate our files and run our checks, we use version 202507.0 of mCRL2's toolset (4). We run this on a machine running Windows 11 with a 7800x3D CPU (8 cores/16 threads) and 32 GB of RAM.

How do we generate the LPS, LTS and PBES files?

We take our specifications for N=2, 3 and 4, for both the fault-sensitive and fault-tolerant algorithms. We use the *corrected* specifications as discussed in Section 5. (See Section 7.1 for a discussion regarding this correction.) These specifications are stored in *.mcrl* files. We adjust the `MSG_LIMIT_GLOBAL`, `MSG_LIMIT_GLOBAL` and `MAX_N_CRASHES` variables to adjust M, S, and C respectively.

We create two specification files for each N: one where the actions `reportActive`, `reportSeq`, and `reportErrorIDinCrashed` (in the fault-tolerant specifications) are enabled, and one specification where these actions are disabled (the 'reduced' model). As explained in Section 2.2.3.2, we cannot check formulas using the *mu X. ...* operator when the model contains these self-loops.

From these *.mcrl* files, we generate the respective linear process specifications (*.lps* files) with mCRL2's *mcrl22lps* tool, with the *regular2* lineariser option.[1]

With each of these *.lps* files, we generate the state space in a linear transition system (*.lts* file) using the *lps2lts* tool, where we use the *cached* option and use up to 8 threads. We set the *–no-info* option to reduce the size of the *.lts* files.

For each of these *.lts* files, we use the *ltsconvert* tool to reduce the state space further, with the options *–no-reach* and *–no-state* set. Specifically, using the *–equivalence=bisim* option, we generate equivalent LTS files that preserve strong bisimilarity (22). By working from these reduced *_bisim.lts* files, we get considerably smaller *.pbes* files to check.[2] We have provided the script *ConvertLts.py* (58) which can automatically carry out this conversion.

As discussed previously, certain properties can only be checked in certain models. As an example, we do not check properties related to crashes in the fault-*sensitive* models, and we only check formulas using the *mu X.* operator on our *reduced* models. Table 6.1 shows the properties that we check for each type of model.

	Normal model	Reduced model
Fault-sensitive	P1, P2, P4, P5, P7, P11, Safety	P0, Liveness
Fault-tolerant	P1, P2, P3, P4, P5, P7, P8, P9, P12, Safety	P0, P6, P10, Liveness

Table 6.1: The properties that we check for each normal/reduced model, for both the fault-sensitive and fault-tolerant models.

For each *_bisim.lts* file and for each property we wish to check on that model, we use the *lts2pbes* tool, using the option *–preprocess-modal-operators*, to create a *.pbes* file which we can check. If we want a counter-example that shows why a certain property does *not* hold, we can additionally set the *–counter-example* option.[3] Our script *CreatePbes.py* (58) can automatically create the *.pbes* files from our *_bisim.lts* files.

Finally, for each *.pbes* file, we use the *pbessolve* tool to whether the property holds for the given model. We again use up to 8 threads, and we use the default solve-strategy. Using the script *CheckPbes.py* (58), we can automatically run the *pbessolve* tool on each

[1] mCRL2's documentation suggests enabling the *–no-globvars* option when using *regular2*. On initial testing, this option does not appear to have a significant impact on either the run-time or memory usage when checking the resulting *.pbes* files, nor on the resulting *.lts* files.

[2] We did also check the properties by creating *.pbes* files from the original, non-converted *.lts* files. The results were unchanged.

[3] We generally do **not** include the counter-example information in our *.pbes* files, to reduce the size of the files.

of our properties.[1]

By default, the *lps2lts* tool generates the entire state space. Where this is not possible, we can instead use the highway search technique (17) to generate a randomised subset of the state space. When working with highway search, we set the size of the todo list at 1000 for $N = 3$ and 10000 for $N = 4$ with the option *–todo-max=...*, and we set an explicit maximum size of the *.lts* file of ten million states with the *–max=10000000* option. Once we have the *.lts* files, we continue *ltsconvert*. Because we only have a *subset* of the state space, we cannot verify liveness properties.[2] Instead, we focus only on checking the Safety property for these subset-models.

6.2 Model sizes

In Tables 6.2 and 6.3, we show the number of states in the models we create from the fault-sensitive specification, for different values of N, S and M:

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	632	3006	840	4968
3	7127	82451	8274	100581
4	46891	996544	51125	1098891

Table 6.2: The number of states for our normal fault-sensitive models.

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	604	2856	800	4687
3	6854	78711	7965	95995
4	45448	957066	49602	1056063

Table 6.3: The number of states for our reduced fault-sensitive models.

Similarly, Tables 6.4 and 6.5 show the size of the fault-tolerant models, where we fix $C = N - 1$:

[1] When using our *CheckPbes.py* script, we actually only use 4 threads. It appears that we can use no more than 4 threads when using our scripts, while through *mcrl2-gui* we were able to use at least 8 threads. Using fewer threads for *pbessolve* did not appear to have a major impact on the runtime.

[2] We cannot check a property "Eventually, X always happens" if we only have a subset of the state space, as X might occur in part of the state space we have not covered.

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	2594	11024	3842	20712
3	335007	3274238	639183	8222201
4	43993937	-	-	-

Table 6.4: The number of states for our normal fault-tolerant models, with $C = N - 1$.

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	2491	10665	3712	20079
3	322690	3160864	622087	8007417
4	42429949	-	-	-

Table 6.5: The number of states for our reduced fault-tolerant models, with $C = N - 1$.

For $N = 4$, we were unable to generate the state space, and abandoned generating the model for S=1 & M=2 after letting *lps2lts* run for over an hour.

In Tables 6.6 and 6.7, we show the model sizes after creating equivalent models with the *ltsconvert* tool. Compared to Tables 6.2 and 6.4, we see a considerable reduction in the number of states.

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	406	567	1646	2568
3	4039	4760	34625	41825
4	26021	28207	381931	414632

Table 6.6: The number of states for our equivalent (strongly bisimilar) normal fault-sensitive models.

N	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
2	1349	4569	2033	7905
3	108704	814925	200091	1985025
4	7165489	-	-	-

Table 6.7: The number of states for our equivalent (strongly bisimilar) normal fault-tolerant models, with $C = N - 1$.

Finally, in Table 6.8, we fix S and M at $S = M = 1$, and show the size of the normal fault-tolerant models for varying C:

N	C=0	C=1	C=2	C=3
2	840	2594	-	-
3	10831	155048	335007	-
4	77790	4069000	25184129	43993937

Table 6.8: The number of states for our normal fault-tolerant models, with $S = M = 1$ and with different values C. Note that we require that $0 \leq C \leq N - 1$.

A note on the differing amount of states between the normal & reduced models

In Tables 6.3 and 6.5, we see that each reduced model has slightly fewer states than in the corresponding normal model in Tables 6.2 and 6.4. This was not expected, as the only difference between the models is that we disable some `reportX` actions, where the state should not be altered.

Analysing the model with the *ltsgraph* tool, as well as the underlying specification with the *LPSXSim* tool[1], we find that some variables in a node are not immediately initialised. Specifically, the lists, such as the `mCtrList` list of counters at each node, are initially empty lists. After the first action that a node takes, these are then properly initialised. As such, if a node does a `reportX` action as its first action, the transition creates a *new* state, where the lists are properly initialised. Following this, any further `reportX` actions taken by this node then form a self-loop to the current state.

As the reduced models do not have these `reportX` actions, these models do not contain these (effectively) duplicate states.

6.2.1 Larger models with highway search

To check some larger models, we use the highway search (17) option in *lps2lts*, allowing us to generate a randomised subset of the total state space.

For $N = 4$, because we could not check the entire model for $S = M = 1$, we generate three files containing a subset of the state space. Because highway search is randomised, it is likely that these 3 files contain (some, but not all) overlapping states, and we have no explicit guarantee of coverage. These files consist (respectively) of 1302696, 2953109 and 816047 states, and we found that the Safety property held for each of the files.

[1]The *LPSXSim* tool allows us to simulate a trace through our model. From the initial state, we can take actions, and observe the values of specific variables. The tool also provides a *random play* option, which proved useful while debugging the model specifications.

Next, we set out to check some larger instances, again using highway search. For $N = 3$ and $N = 4$, we set $S = M = 100$ and $C = N - 1$, and generated three files per value of N of the fault-tolerant specification, each consisting of approximately 10 million states.

Again, we use *pbessolve* and find that the Safety property holds for each of these six files, containing a subset of the larger state space.

6.3 Results from checking properties

We found that all our checked properties hold for our $N = 2$, $N = 3$ and $N = 4$ fault-sensitive models. As we discuss in Section 7.1, the Safety property does not hold on the fault-tolerant algorithm as originally set out in (37). With the change applied as specified in 7.1, we created *corrected* specifications, and generated our fault-tolerant models. We found that each of our properties hold for the $N = 2$ and $N = 3$ models.

We were unable to check properties for a complete tolerant model for $N = 4$, due to memory constraints.[1]

When running *pbessolve* on the Safety and Liveness property *.pbes* files, we find that it takes ≤ 1 second to check the smaller instances (e.g. $N = 2$), up to a couple of minutes for the larger instances, with the Safety property for the $N = 3$, $S = 2$, $M = 2$ and $C = 2$ fault-tolerant model taking approximately 3.5 minutes to check. See Tables 6.9 and 6.10.

Property	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
Safety	0.484	3.069	0.541	3.644
Liveness	0.375	1.268	0.344	1.308

Table 6.9: The wall clock time in seconds, averaged across 5 runs, to check the Safety and Liveness properties for the fault-sensitive $N = 3$ models using *pbessolve* with 4 threads.

Property	S=1, M=1	S=1, M=2	S=2, M=1	S=2, M=2
Safety	9.444	81.742	17.365	207.567
Liveness	3.078	27.056	5.601	69.404

Table 6.10: The wall clock time in seconds, averaged across 5 runs, to check the Safety and Liveness properties for the fault-tolerant $N = 3$ models, with $C = N - 1$, using *pbessolve* with 4 threads.

[1] As we worked on Windows, the *pbessolvesymbolic* tool was unavailable to us.

7

Discussion

7.1 Bug in NewSuccessor$_i$

As mentioned in Section 2.1.3.3, the fault-tolerant algorithm draws a distinction between the $Crashed_i$ and $Report_i$ sets of nodes, and specifically allows the nodes i to receive basic messages from crashed nodes $j \in Report_i$, as long as $j \notin Crashed_i$.

If the basic algorithm has not yet terminated when some node j crashes, the algorithm generally ensures that the token will pass every node at least one more time, so that each node is informed of j's crash.[1] Specifically, this process generally ensures that j is put into every node i's $Crashed_i$ set, and removes j from $Report_i$ if i detected j's crash.

However, there is one exception: in NewSuccessor$_i$ 9, if all nodes bar i have crashed (so $next_i = i$), then we can Announce here (Line 6 of NewSuccessor$_i$), instead of in ReceiveToken$_i$ 10. In this case, we do *not* have the same assurances over $Report_i$ and $Crashed_i$.

Using NewSuccessor$_i$, we can violate the Safety property. Take $N = 2$, $M = 2$, $S = 1$, $C = 1$. The following execution then violates the property:

1. Node 1 sends a basic message m to node 0.

2. Node 1 crashes.

3. Node 0 becomes passive.

4. Node 0 runs HandleToken$_0$, and sends the token to node 1.

[1] If the basic algorithm *has* already terminated, then a node may correctly be able to detect this termination even without knowing of j's crash.

5. Node 0 detects node 1's crash. $Report_0 = \{1\}$. $next_0 = 1$, so node 0 moves to NewSuccessor$_0$.

6. In NewSuccessor$_0$, node 0 finds that the new $next_0$ is 0. Node 0 is passive, so node 0 now Announces.

7. The basic message m from node 1 arrives at node 0. Although 1 *is* in $Report_0$, 1 is *not* in $Crashed_0$. As such, node 0 can run the full ReceiveBasicMessage$_0(m, 1)$ procedure 7.
   Node 0 is passive, and receives a basic message, so node 0 becomes active.

Node 0 becomes active *after* detecting termination, thereby violating the Safety property. We can also show this violation in mCRL2: in (58), we provide the *Tol_Bug_Demonstration* folder. This contains an **un**corrected specification for $N = 2$, as well as the resulting *.lps* and *.lts* files. From this *.lts* file, we generated a *.pbes* to check the Safety property, where we included the counterexample information. Using *pbessolve*, we found that the Safety property did not hold for this model, and it provides a counter-example trace in the model, similar to the execution we provided above.

The solution to this bug is straight-forward: before we do Announce in NewSuccessor$_i$, we must *first* update $Crashed_i$ and $Report_i$[1], as we show in Algorithm 11:

Algorithm 11 *Corrected* NewSuccessor$_i$

Require: $\exists j \in [0..N-1].j \notin Crashed_i \cup Report_i$

1: $next_i \leftarrow (next_i + 1)\%N$; ▷ Initially, $next_i \in Report_i \cup Crashed_i$
2: **while** $next_i \in Crashed_i \cup Report_i$ **do**
3: $\quad next_i \leftarrow (next_i + 1)\%N$;
4: **if** $next_i = i$ **then**
5: $\quad wait(passive_i)$;
6: $\quad Crashed_i \leftarrow Crashed_i \cup Report_i$; $\quad Report_i \leftarrow \emptyset$; ▷ Our changes
7: $\quad$ Announce;
8: **if** $black_i \neq i$ **then**
9: $\quad black_i \leftarrow furthest_i(black_i, next_i)$;

An alternative solution would be to adjust ReceiveBasicMessage 7, such that we only receive basic messages from nodes $j \notin Crashed_i \cup Report_i$, instead of only $j \notin Crashed_i$.

[1]Strictly speaking, it does not appear to be necessary to empty $Report_i$. Instead, we do this to preserve the (unverified) invariant $Crashed_i \cap Report_i = \emptyset$.

We believe the fix put forward in Algorithm 11 is more desirable, as we force the basic algorithm to discard fewer basic messages.

Our model specifications, as set out in Section 5.2, uses our corrected definition for NewSuccessor$_i$ 11. In `NewSucc_4_*`, we include the updates to $Crashed_i$ and $Report_i$ as set out in Line 6. With this correction applied, the Safety property *does* hold for the instances we check. (As discussed in Section 6.3.)

7.2 Model sizes

As you can see in the tables in Section 6.2, even the smallest instances of our models consist of hundreds or thousands of states. In Figure 7.1, we show the tolerant model with $N = 2$ and $S = M = C = 1$, displayed by mCRL2's *ltsgraph* tool.

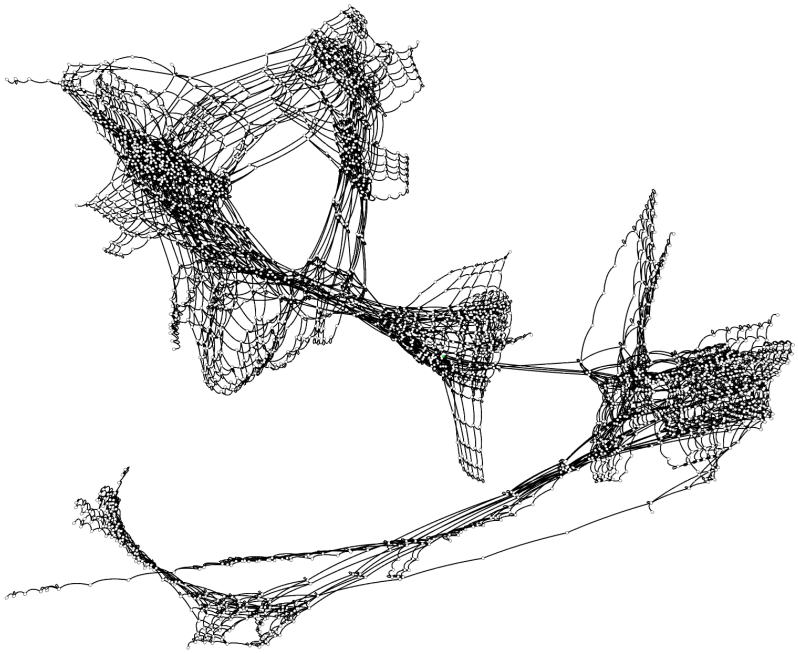

Figure 7.1: The $N = 2$ & $S = M = C = 1$ fault-tolerant model, displayed by *ltsgraph*.

Manually examining all 2491 states for even this (relatively) small model is practically infeasible, hence why we check our properties with *pbessolve*.

7.2.1 State space explosion

Altering S and M

Looking at Tables 6.2-6.5, we find that increasing S causes a relatively small increase in

the size of the models, compared to increasing M. This is expected. When we increase M, it means that *any* node can send out an extra message, meaning N different nodes could (assuming they are/become active) send out this extra message. In contrast, an increase in S merely allows for one additional, with a fixed order, loop of the token through the ring.

Altering N

Next, in the fault-sensitive models, we see that increasing N increases the model size by an order of magnitude of 10. For the same N, S and M, we see that the fault-tolerant models are considerably bigger, and they grow by an order of magnitude of 100 when we increase N.

The addition of crashes, as well as the changes made in the underlying algorithm, will largely explain this. We add an extra option to many of the states, namely the option for a node to crash, which creates a state that would not have existed in the fault-sensitive model.[1] In Tables 6.4 and 6.5, we fix $C = N - 1$. Thus, when we increase N, we also increase the number of crashes in the model.

Altering C

Finally, in Table 6.8, we maintain $S = M = 1$, and alter C. We see that going from $C = N - 2$ to $C = N - 1$ adds *relatively* few states. For $N = 3$ and 4, we approximately double the number of states when going from $C = N - 2$ to $C = N - 1$, while going from e.g. $C = 0$ to $C = 1$ causes an order of magnitude increase in state size. We believe that there are two main reasons for this last crash adding relatively few extra states. Firstly, in the states where $N - 2$ crashes have already occurred, we have relatively few options, as we only have 2 non-crashed nodes remaining. Secondly, when we add the $N - 1$'th crash, the final remaining node will not continue with the 'normal' operation of the algorithm, instead quickly detecting termination and Announcing through NextSuccessor$_i$ 11.

Using *ltsconvert*

With the *ltsconvert* tool, we obtain considerably smaller state spaces, as shown in Tables 6.6 and 6.7 when compared to Tables 6.2 and 6.4. Using these smaller state spaces, we found that generating and checking the *.pbes* files was considerably faster. As an example: checking the Liveness *.pbes* file for the fault-tolerant model with $N = 3$, $M = 2$, $S = 1$,

[1] In fact, if the first action done from the initial state is to have a node crash, we effectively are left with the $N - 1$ and $C - 1$ model from this state.

$C = N - 1$ based on the original *.lts* file takes approximately 2 minutes. After using *ltsconvert*, it only takes 5 seconds to check the smaller *.pbes* file. (See Table 6.10.)

Limiting factors

In terms of generating state spaces and checking properties, we encountered two main limitations.

First, as mentioned in Section 6.2, generating large models, such as $S = 1$ & $M = 2$ for $N = 4$, may simply take too long to complete. This is despite the fact that, looking at Table 6.10, we likely *would* have been able to generate and check the resulting *.pbes* files if we were able to generate the model.

Secondly, and related to the first, we ran out of memory when generating and checking some *.pbes* files, such as when generating *.pbes* files for $N = 4$. In fact, if we generate the *.pbes* for the Safety property in $N = 3$, $M = 2$, $S = 2$ without the *–no-info* option and without using *ltsconvert* first, the *pbessolve* tool exceeds the available memory on our machine.

7.3 Correctness of the algorithms

As mentioned in Section 6.3, we found that all of our properties held for each of our models. What claims can we now make about our two algorithms, the fault-sensitive (12) and fault-tolerant (37) versions of Safra's algorithm?

We turn back to the two main properties set out in (37): Safety, or that *if* termination is detected, indeed all nodes are passive/crashed and do not become active, and Liveness, or that the network as a whole eventually detects termination somewhere once the basic algorithm has terminated.

We have verified that the Safety property holds for our complete models, as well as for our various subset-models.

For the Liveness property, we show our paper proof (sketch) of Theorems 3 and 4, and we show with properties **P11** and **P12** that these bounds also hold for our models. These theorems show that, once the basic algorithm terminates, the token will complete a finite number of iterations, before some node is guaranteed to detect termination. On top of this, we combine our **Liveness** property, which relies on the `basicAlgoDone` action, with the arguably stronger **P0**, which guarantees that all traces in the model eventually reach Announce, and **P6**, which guarantees that a *different* node will always eventually Announce

if a node i first Announces and then crashes, properties, from which we can conclude that Liveness is guaranteed in our finite models.

We see no reason why these results would not extend to larger instances. The fault-tolerant algorithm has, to our knowledge, no limit to the number of crashes it can handle[1], including multiple *concurrent* crashes. The fault-tolerant algorithm has its special case in NewSuccessor$_i$ 11 when $N-1$ nodes have crashed. Outside of this, the algorithm does not seemingly change if we have (for large N) one crash or a hundred. The Safety property holding for multiple subset-models with large M, S also indicates that a larger number of basic messages or iterations around the ring is unlikely to cause any issues not found in our smaller models.

Further, we note that similar works, such as (26, 28, 56), have also been limited to checking instances of no more than $N = 3$ or $N = 4$.

As such, we consider that the fault-sensitive and fault-tolerant versions of Safra's algorithm, as laid out in (37) and as corrected in Section 7.1, are formally verified to be correct.

[1]Our models require at least 1 node to not crash, in order to actually have a node perform the *Announce* action. If all nodes crash, termination detection is trivial.

8

Conclusion

In this work, we analysed a fault-sensitive (12) and fault-tolerant (37) version of Safra's algorithm. We provided a paper proof for the liveness property for the fault-sensitive algorithm, and provided a sketch towards the same proof for the fault-tolerant algorithm.

We then created formal specifications based on both algorithms, and model checked a number of properties on a series of different models created from these specifications.

We encountered a bug in the fault-tolerant algorithm, and put forward a solution to fix this bug.

After updating the fault-tolerant specification, we found that all of our properties held on both the fault-sensitive and fault-tolerant models.

Combining our various approaches, we have shown that the two algorithms are now likely free of bugs, under the assumptions stated in (37).

For future work, this work can be replicated for *other* distributed algorithms. As an example, (37) proposes another fault-tolerant algorithm based on Safra's algorithm, this time making use of stable storage. We imagine that it would be relatively straightforward to adapt our models, to model check this alternative algorithm.

References

[1] Muhammad Atif and Jan Friso Groote. 2023. *Understanding behaviour of distributed systems using mCRL2* (2023 ed.). Springer International Publishing, Cham, Switzerland. https://doi.org/10.1007/978-3-031-23008-0 23

[2] Gerd Behrmann, Alexandre David, and Kim G. Larsen. 2004. *A Tutorial on Uppaal*. Springer Berlin Heidelberg, Berlin, Heidelberg, 200–236. https://doi.org/10.1007/978-3-540-30080-9_7 30

[3] Parth Bora, Pham Duc Minh, and Tim A.C. Willemse. 2024. Modelling the Raft Distributed Consensus Protocol in mCRL2. *Electronic Proceedings in Theoretical Computer Science* 399 (March 2024), 7–20. https://doi.org/10.4204/eptcs.399.4 30

[4] Olav Bunte, Jan Friso Groote, Jeroen J. A. Keiren, Maurice Laveaux, Thomas Neele, Erik P. de Vink, Wieger Wesselink, Anton Wijs, and Tim A. C. Willemse. 2019. The mCRL2 Toolset for Analysing Concurrent Systems. In *Tools and Algorithms for the Construction and Analysis of Systems*, Tomáš Vojnar and Lijun Zhang (Eds.). Springer International Publishing, Cham, 21–39. https://doi.org/10.1007/978-3-030-17465-1_2 1, 2, 18, 19, 23, 27, 29, 30, 31, 83

[5] Ignacio Cano, Markus Weimer, Dhruv Mahajan, Carlo Curino, and Giovanni Matteo Fumarola. 2016. Towards Geo-Distributed Machine Learning. https://doi.org/10.48550/arXiv.1603.09035 arXiv:1603.09035 [cs.LG] 3

[6] Tushar Deepak Chandra and Sam Toueg. 1996. Unreliable failure detectors for reliable distributed systems. *J. ACM* 43, 2 (March 1996), 225–267. https://doi.org/10.1145/226643.226647 6

[7] K. Mani Chandy and Leslie Lamport. 1985. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Trans. Comput. Syst.* 3, 1 (1985), 63–75. https://doi.org/10.1145/214451.214456 3, 5

[8] Franco Cicirelli, Libero Nigro, and Paolo F. Sciammarella. 2016. Model Checking Mutual Exclusion Algorithms Using Uppaal. In *Software Engineering Perspectives and Application in Intelligent Systems*, Radek Silhavy, Roman Senkerik, Zuzana Kominkova Oplatkova, Petr Silhavy, and Zdenka Prokopova (Eds.). Springer International Publishing, Cham, 203–215. https://doi.org/10.1007/978-3-319-33622-0_19 31

[9] Edmund M. Clarke. 1997. Model checking. In *Foundations of Software Technology and Theoretical Computer Science*, S. Ramesh and G. Sivakumar (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 54–56. https://doi.org/10.1007/BFb0058022 18

[10] Edmund M. Clarke, William Klieber, Miloš Nováček, and Paolo Zuliani. 2012. *Model Checking and the State Explosion Problem.* Springer Berlin Heidelberg, Berlin, Heidelberg, 1–30. https://doi.org/10.1007/978-3-642-35746-6_1 19

[11] Giorgio Delzanno, Michele Tatarek, and Riccardo Traverso. 2014. Model Checking Paxos in Spin. *Electronic Proceedings in Theoretical Computer Science* 161 (Aug. 2014), 131–146. https://doi.org/10.4204/eptcs.161.13 31

[12] Murat Demirbas and Anish Arora. 2000. *An optimal termination detection algorithm for rings.* Technical Report. The Ohio State University. 1, 5, 6, 7, 29, 30, 33, 35, 38, 39, 45, 93, 95

[13] Edsger W. Dijkstra. 1962 or 1963. Over de sequentialiteit van procesbeschrijvingen. (1962 or 1963). 31

[14] Edsger W. Dijkstra. 1965. Solution of a problem in concurrent programming control. *Commun. ACM* 8, 9 (Sept. 1965), 569. https://doi.org/10.1145/365559.365617 31

[15] Edsger W. Dijkstra. 1987. Shmuel Safra's version of termination detection. (Jan. 1987). http://www.cs.utexas.edu/users/EWD/ewd09xx/EWD998.PDF circulated privately. 5, 29, 30

[16] Edsger W. Dijkstra and C.S. Scholten. 1980. Termination detection for diffusing computations. *Inform. Process. Lett.* 11, 1 (1980), 1–4. https://doi.org/10.1016/0020-0190(80)90021-6 4

[17] Tom A.N. Engels, Jan Friso Groote, Muck J. van Weerdenburg, and Tim A.C. Willemse. 2009. Search algorithms for automated validation. *The Journal of Logic and Algebraic Programming* 78, 4 (2009), 274–287. https://doi.org/10.1016/j.jlap.2008.11.003 IFIP WG1.8 Workshop on Applying Concurrency Research in Industry. 19, 85, 87

[18] W. H. J. Feijen and A. J. M. van Gasteren. 1999. *Shmuel Safra's Termination Detection Algorithm*. Springer New York, New York, NY, 313–332. https://doi.org/10.1007/978-1-4757-3126-2_29 5, 29, 30

[19] Wan Fokkink. 2018. *Distributed algorithms: an intuitive approach* (2nd ed.). MIT Press. 5, 29

[20] Nissim Francez. 1980. Distributed Termination. *ACM Trans. Program. Lang. Syst.* 2, 1 (Jan. 1980), 42–55. https://doi.org/10.1145/357084.357087 4

[21] Hubert Garavel, Frédéric Lang, Radu Mateescu, and Wendelin Serwe. 2011. CADP 2010: A Toolbox for the Construction and Analysis of Distributed Processes. In *Tools and Algorithms for the Construction and Analysis of Systems*, Parosh Aziz Abdulla and K. Rustan M. Leino (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 372–387. https://doi.org/10.1007/978-3-642-19835-9_33 29

[22] Jan Friso Groote and David N. Jansen. 2025. A State-Based O(m log n) Partitioning Algorithm for Branching Bisimilarity. In *36th International Conference on Concurrency Theory (CONCUR 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 348)*, Patricia Bouyer and Jaco van de Pol (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 18:1–18:16. https://doi.org/10.4230/LIPIcs.CONCUR.2025.18 84

[23] Jan Friso Groote and Jeroen J. A. Keiren. 2021. Tutorial: Designing Distributed Software in mCRL2. In *Formal Techniques for Distributed Objects, Components, and Systems*, Kirstin Peters and Tim A. C. Willemse (Eds.). Springer International Publishing, Cham, 226–243. https://doi.org/10.1007/978-3-030-78089-0_15 18, 19

[24] Jan Friso Groote, Tim W.D.M. Kouters, and Ammar Osaiweran. 2015. Specification guidelines to avoid the state space explosion problem. *Software Testing, Verification and Reliability* 25, 1 (2015), 4–33. https://doi.org/10.1002/stvr.1536 19, 23, 47

[25] Jan Friso Groote and Mohammad Reza Mousavi. 2014. *Modeling and Analysis of Communicating Systems*. MIT Press. https://doi.org/10.7551/mitpress/9946.001.0001 1, 19, 20, 24, 27

[26] Yousra Hafidi, Jeroen J. A. Keiren, and Jan Friso Groote. 2021. Fair Mutual Exclusion for N Processes (extended version). *CoRR* abs/2111.02251 (2021), 18 pages. https://doi.org/10.48550/arXiv.2111.02251 arXiv:2111.02251 31, 94

[27] Reiner Hähnle and Marieke Huisman. 2019. *Deductive Software Verification: From Pen-and-Paper Proofs to Industrial Tools*. Springer International Publishing, Cham, 345–373. https://doi.org/10.1007/978-3-319-91908-9_18 18

[28] Imran Riaz Hasrat, Muhammad Atif, and Muhammad Naeem. 2018. Formal Specification and Analysis of Termination Detection by Weight-throwing Protocol. *International Journal of Advanced Computer Science and Applications* 9, 4 (2018), 269–282. https://doi.org/10.14569/IJACSA.2018.090441 30, 33, 35, 94

[29] J.-M. Helary, M. Hurfin, A. Mostefaoui, M. Raynal, and F. Tronel. 2000. Computing global functions in asynchronous distributed systems with perfect failure detectors. *IEEE Transactions on Parallel and Distributed Systems* 11, 9 (2000), 897–909. https://doi.org/10.1109/71.879773 10

[30] Matthew Hennessy and Robin Milner. 1985. Algebraic laws for nondeterminism and concurrency. *J. ACM* 32, 1 (Jan. 1985), 137–161. https://doi.org/10.1145/2455.2460 24

[31] Gerard J. Holzmann. 1997. The model checker SPIN. *IEEE Transactions on Software Engineering* 23, 5 (1997), 279–295. https://doi.org/10.1109/32.588521 29

[32] Gerard J. Holzmann. 1998. An Analysis of Bitstate Hashing. *Formal methods in system design* 13, 3, 289–307. https://doi.org/10.1023/A:1008696026254 19

[33] Heidi Howard and Richard Mortier. 2020. Paxos vs Raft: have we reached consensus on distributed consensus?. In *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data* (Heraklion, Greece) *(PaPoC '20)*. Association for

Computing Machinery, New York, NY, USA, Article 8, 9 pages. https://doi.org/10.1145/3380787.3393681 30

[34] S.-T. Huang. 1989. Detecting termination of distributed computations by external agents . In *Proceedings. The 9th International Conference on Distributed Computing Systems*. IEEE Computer Society, Los Alamitos, CA, USA, 79,80,81,82,83,84. https://doi.org/10.1109/ICDCS.1989.37933 29

[35] Muhammad Ali Ismail, Dr. Shahid H. Mirza, and Dr. Talat Altaf. 2011. A Parallel and Concurrent Implementation of Lin-Kernighan Heuristic (LKH-2) for Solving Traveling Salesman Problem for Multi-Core Processors using SPC3 Programming Model. *International Journal of Advanced Computer Science and Applications* 2, 7 (2011), 34–43. https://doi.org/10.14569/IJACSA.2011.020706 3

[36] Xiaojun Ji and Lihua Song. 2016. Mutual Exclusion Verification of Peterson's Solution in Isabelle/HOL. In *2016 Third International Conference on Trustworthy Systems and their Applications (TSA)*. IEEE, 81–86. https://doi.org/10.1109/TSA.2016.22 31

[37] Georgios Karlos, Wan Fokkink, and Per Fuchs. 2021. Fault-Tolerant Termination Detection with Safra's Algorithm. In *Networked Systems - 9th International Conference, NETYS 2021, Virtual Event, May 19-21, 2021, Proceedings (Lecture Notes in Computer Science, Vol. 12754)*, Karima Echihabi and Roland Meyer (Eds.). Springer, 71–87. https://doi.org/10.1007/978-3-030-91014-3_5 1, 4, 5, 9, 10, 11, 17, 29, 30, 33, 35, 40, 42, 45, 57, 70, 78, 88, 93, 94, 95

[38] Donald E. Knuth. 1966. Additional comments on a problem in concurrent programming control. *Commun. ACM* 9, 5 (May 1966), 321–322. https://doi.org/10.1145/355592.365595 31

[39] Igor Konnov, Markus Kuppe, and Stephan Merz. 2022. Specification and Verification with the TLA+ Trifecta: TLC, Apalache, and TLAPS. In *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer International Publishing, Cham, 88–105. https://doi.org/10.1007/978-3-031-19849-6_6 4, 19, 30, 33, 35, 36

[40] Markus Alexander Kuppe, Leslie Lamport, and Daniel Ricketts. 2019. The TLA+ Toolbox. *Electronic Proceedings in Theoretical Computer Science* 310 (Dec. 2019), 50–62. https://doi.org/10.4204/eptcs.310.6 18, 29, 30, 32

[41] L. Lamport. 1977. Proving the Correctness of Multiprocess Programs. *IEEE Transactions on Software Engineering* SE-3, 2 (1977), 125–143. https://doi.org/10.1109/TSE.1977.229904 4, 18

[42] Leslie Lamport. 1998. The part-time parliament. *ACM Trans. Comput. Syst.* 16, 2 (May 1998), 133–169. https://doi.org/10.1145/279227.279229 30

[43] Radu Mateescu and Wendelin Serwe. 2013. Model checking and performance evaluation with CADP illustrated on shared-memory mutual exclusion protocols. *Science of Computer Programming* 78, 7 (2013), 843–861. https://doi.org/10.1016/j.scico.2012.01.003 Special section on Formal Methods for Industrial Critical Systems (FMICS 2009 + FMICS 2010) & Special section on Object-Oriented Programming and Systems (OOPS 2009), a special track at the 24th ACM Symposium on Applied Computing. 29, 31

[44] Radu Mateescu and Mihaela Sighireanu. 2003. Efficient on-the-fly model-checking for regular alternation-free mu-calculus. *Science of Computer Programming* 46, 3 (2003), 255–281. https://doi.org/10.1016/S0167-6423(02)00094-1 Special issue on Formal Methods for Industrial Critical Systems. 24

[45] Aad Mathijssen. 2025. Data types for mCRL2. https://mcrl2.org/web/_downloads/4b1f3f29182d9ace80eda873d67c8720/mcrl2data.pdf Accessed on 04-12-2025. 48

[46] Jeff Matocha and Tracy Camp. 1998. A taxonomy of distributed termination detection algorithms. *Journal of Systems and Software* 43, 3 (1998), 207–221. https://doi.org/10.1016/S0164-1212(98)10034-1 29

[47] Friedemann Mattern. 1989. Global quiescence detection based on credit distribution and recovery. *Inform. Process. Lett.* 30, 4 (1989), 195–200. https://doi.org/10.1016/0020-0190(89)90212-3 29

[48] Neeraj Mittal, Felix C. Freiling, S. Venkatesan, and Lucia Draque Penso. 2008. On termination detection in crash-prone distributed systems with failure detectors. *J. Parallel and Distrib. Comput.* 68, 6 (June 2008), 855–875. https://doi.org/10.1016/j.jpdc.2008.02.001 6, 10

[49] Diego Ongaro and John K. Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference,*

USENIX ATC 2014, Philadelphia, PA, USA, June 19-20, 2014, Garth Gibson and Nickolai Zeldovich (Eds.). USENIX Association, 305–319. https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro Accesssed on 21-01-2026.. 30

[50] Lawrence C. Paulson. 1986. Natural deduction as higher-order resolution. *The Journal of Logic Programming* 3, 3 (1986), 237–258. https://doi.org/10.1016/0743-1066(86)90015-4 18

[51] M. Pease, R. Shostak, and L. Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (April 1980), 228–234. https://doi.org/10.1145/322186.322188 5

[52] Gary L. Peterson. 1981. Myths about the mutual exclusion problem. *Inform. Process. Lett.* 12, 3 (1981), 115–116. https://doi.org/10.1016/0020-0190(81)90106-X 31

[53] Gary L. Peterson and Michael J. Fischer. 1977. Economical solutions for the critical section problem in a distributed system (Extended Abstract). In *Proceedings of the Ninth Annual ACM Symposium on Theory of Computing* (Boulder, Colorado, USA) *(STOC '77)*. Association for Computing Machinery, New York, NY, USA, 91–97. https://doi.org/10.1145/800105.803398 31

[54] Glenn Ricart and Ashok K Agrawala. 1981. An optimal algorithm for mutual exclusion in computer networks. *Commun. ACM* 24, 1 (1981), 9–17. https://doi.org/10.1145/358527.358537 31

[55] Ekaterina Sedletsky, Amir Pnueli, and Mordechai Ben-Ari. 2000. Formal Verification of the Ricart-Agrawala Algorithm. In *FST TCS 2000: Foundations of Software Technology and Theoretical Computer Science*, Sanjiv Kapoor and Sanjiva Prasad (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 325–335. https://doi.org/10.1007/3-540-44450-5_26 31

[56] Evgeniy Shishkin. 2017. Construction and formal verification of a fault-tolerant distributed mutual exclusion algorithm. In *Proceedings of the 16th ACM SIGPLAN International Workshop on Erlang* (Oxford, UK) *(Erlang 2017)*. Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/3123569.3123571 31, 94

[57] Andrew S Tanenbaum and Herbert Bos. 2015. *Modern operating systems.* Pearson Education, Inc. 31

[58] Andy S. Tatman. 2026. Formal Verification of Fault-Tolerant Safra's Algorithm - Artifact. https://doi.org/10.17632/tnt3z9tzcy.1
For a smaller set of files, containing only the specifications, formulas, and python scripts, see the GitHub link in Section 6. 2, 45, 57, 78, 83, 84, 90

[59] Yu-Chee Tseng. 1995. Detecting termination by weight-throwing in a faulty distributed system. *J. Parallel Distrib. Comput.* 25, 1 (Feb. 1995), 7–15. https://doi.org/10.1006/jpdc.1995.1025 29, 30

[60] Tatsuhiro Tsuchiya and André Schiper. 2008. Using Bounded Model Checking to Verify Consensus Algorithms. In *Distributed Computing*, Gadi Taubenfeld (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 466–480. https://doi.org/10.1007/978-3-540-87779-0_32 31

[61] D. J. Walker. 1989. Automated analysis of mutual exclusion algorithms using CCS. *Form. Asp. Comput.* 1, 1 (March 1989), 273–292. https://doi.org/10.1007/BF01887209 31

[62] Wieger Wesselink. [n. d.]. lps2lts - mCRL2 202507.0 documentation. https://www.mcrl2.org/web/user_manual/tools/release/lps2lts.html Accessed on 03-01-2026. 43

[63] Junfeng Yang, Tisheng Chen, Ming Wu, Zhilei Xu, Xuezheng Liu, Haoxiang Lin, Mao Yang, Fan Long, Lintao Zhang, and Lidong Zhou. 2009. MODIST: transparent model checking of unmodified distributed systems *(NSDI'09)*. USENIX Association, USA, 213–228. 29

[64] Pierre Castéran Yves Bertot. 2010. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions.* Springer Berlin. https://doi.org/10.1007/978-3-662-07964-5 18, 31